Multimodal Machine Learning

Lecture 2.2: Unimodal Representations (Part 2)

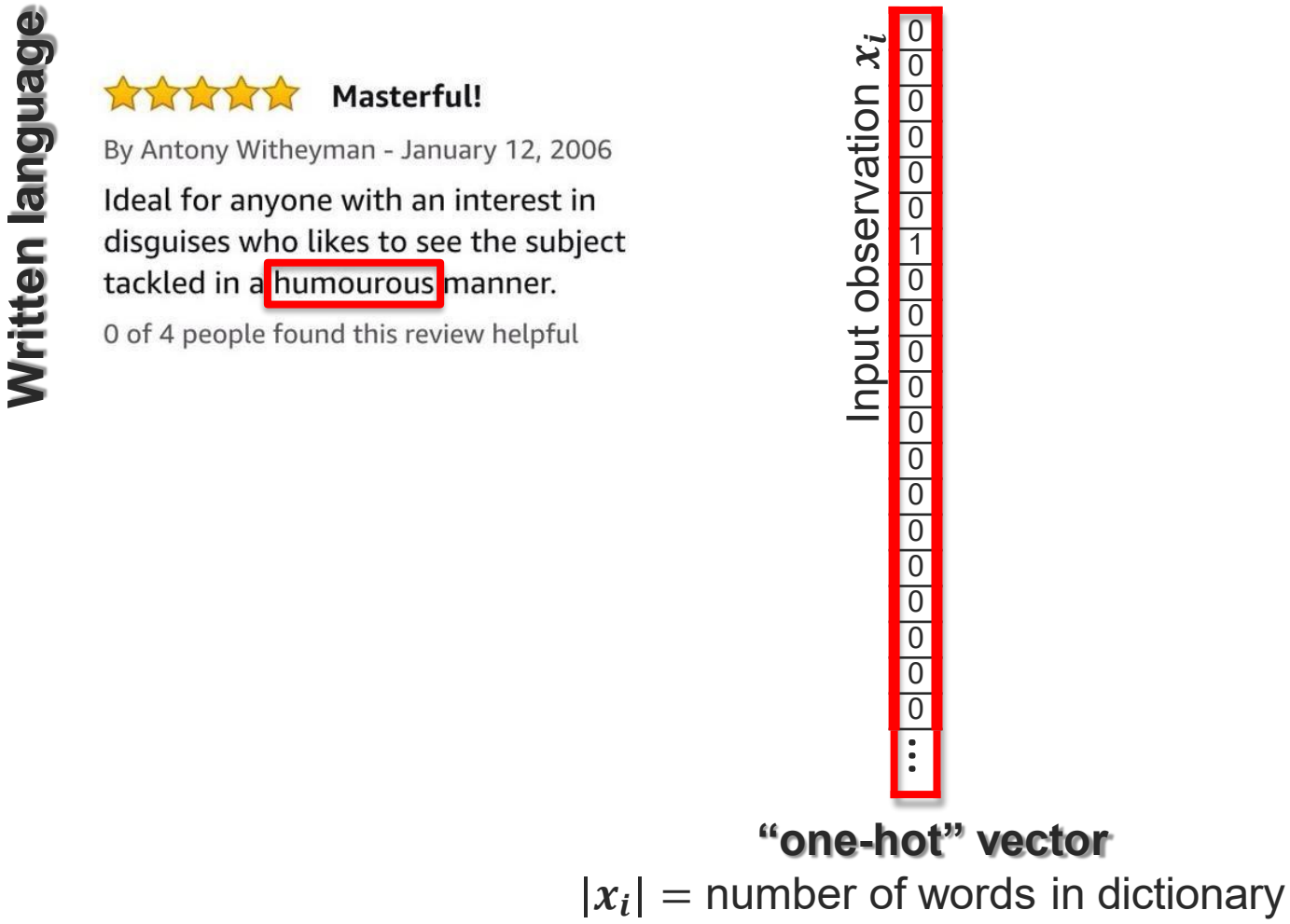
Đàm Quang Tuấn

Lecture Objectives

- Word representations
  - Distributional hypothesis, Word2Vec, and GloVe
- Sequence modeling and language models
  - Vanilla RNNs and backpropagation through time
  - Vanishing gradients; LSTM and GRU
  - Encoder-decoder models, attention, and applications
- Syntax and language structure
  - Phrase-structure and dependency grammars
  - Recursive neural networks / TreeRNN

Word Representations

Simple Word Representation



What is the meaning of “bardiwac”?

- He handed her her glass of bardiwac.
- Beef dishes are made to complement the bardiwacs.
- Nigel staggered to his feet, face flushed from too much bardiwac.
- Malbec, one of the lesser-known bardiwac grapes, responds well to Australia’s sunshine.
- I dined off bread and cheese and this excellent bardiwac.
- The drinks were delicious: blood-red bardiwac as well as light, sweet Rhenish.

$\Rightarrow$ bardiwac is a heavy red alcoholic beverage made from grapes

How to learn (word) features/representations?

- **Distribution hypothesis:** Approximate the word meaning by its surrounding words
- Words used in a similar context will lie close together

Geometric interpretation

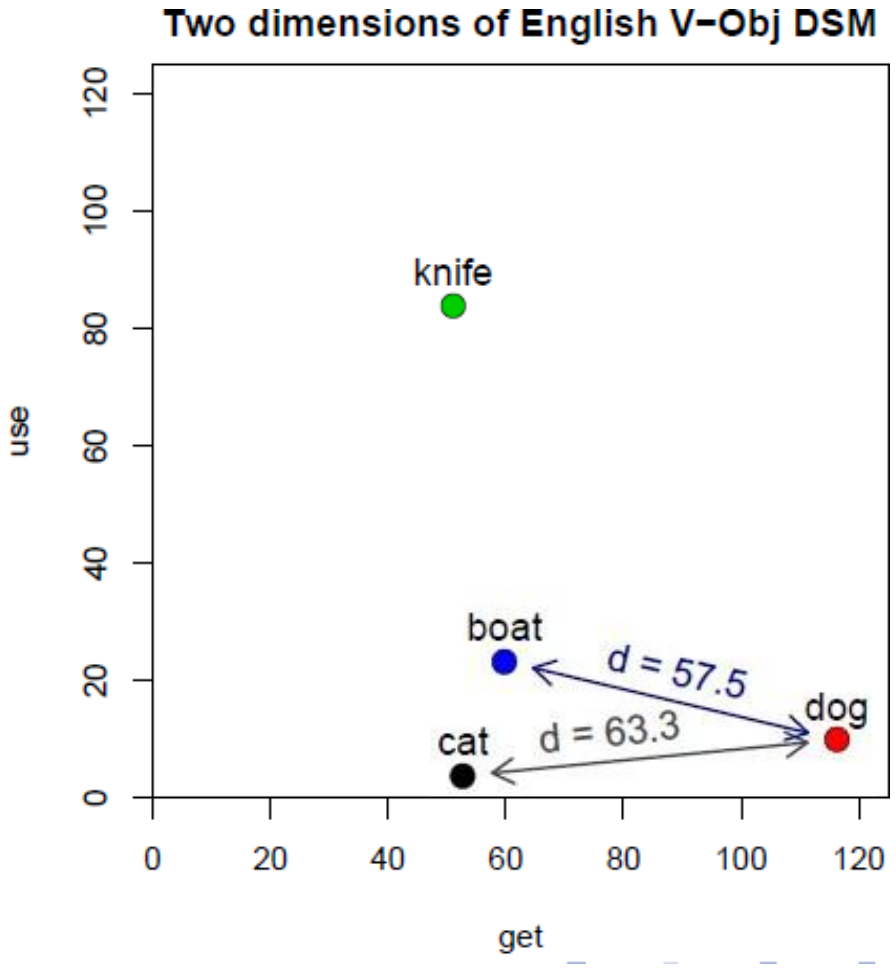
- row vector $\mathbf{x}_{dog}$ describes usage of word *dog* in the corpus
- can be seen as coordinates of point in *n*-dimensional Euclidean space R^n

	get	see	use	hear	eat	kill
knife	51	20	84	0	3	0
cat	52	58	4	4	6	26
dog	115	83	10	42	33	17
boat	59	39	23	4	0	0
cup	98	14	6	2	1	0
pig	12	17	3	2	9	27
banana	11	2	2	0	18	0

co-occurrence matrix M

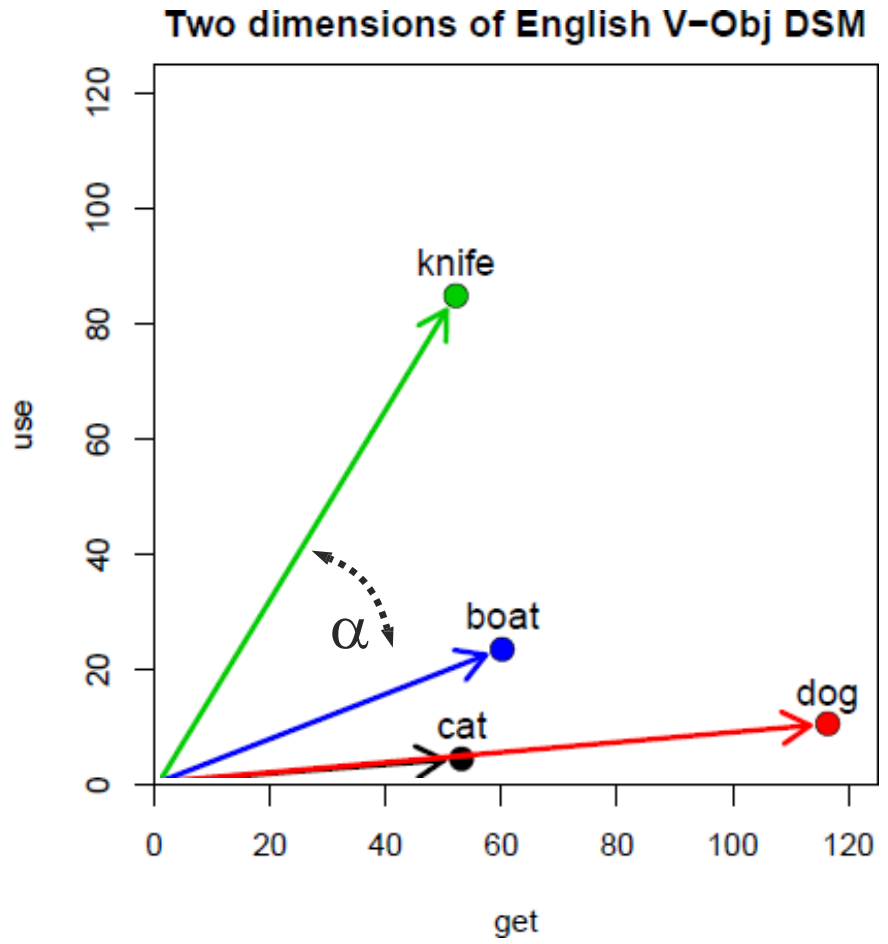
Distance and similarity

- illustrated for two dimensions: *get* and *use*: $\mathbf{x}_{dog}$ = (115, 10)
- similarity = spatial proximity (Euclidean distance)
- location depends on frequency of noun ($f_{dog} \approx 2.7 \cdot f_{cat}$)



Angle and similarity

- direction more important than location
- normalise "length" $||\mathbf{x}_{dog}||$ of vector
- or use angle α as distance measure



How to learn (word) features/representations?

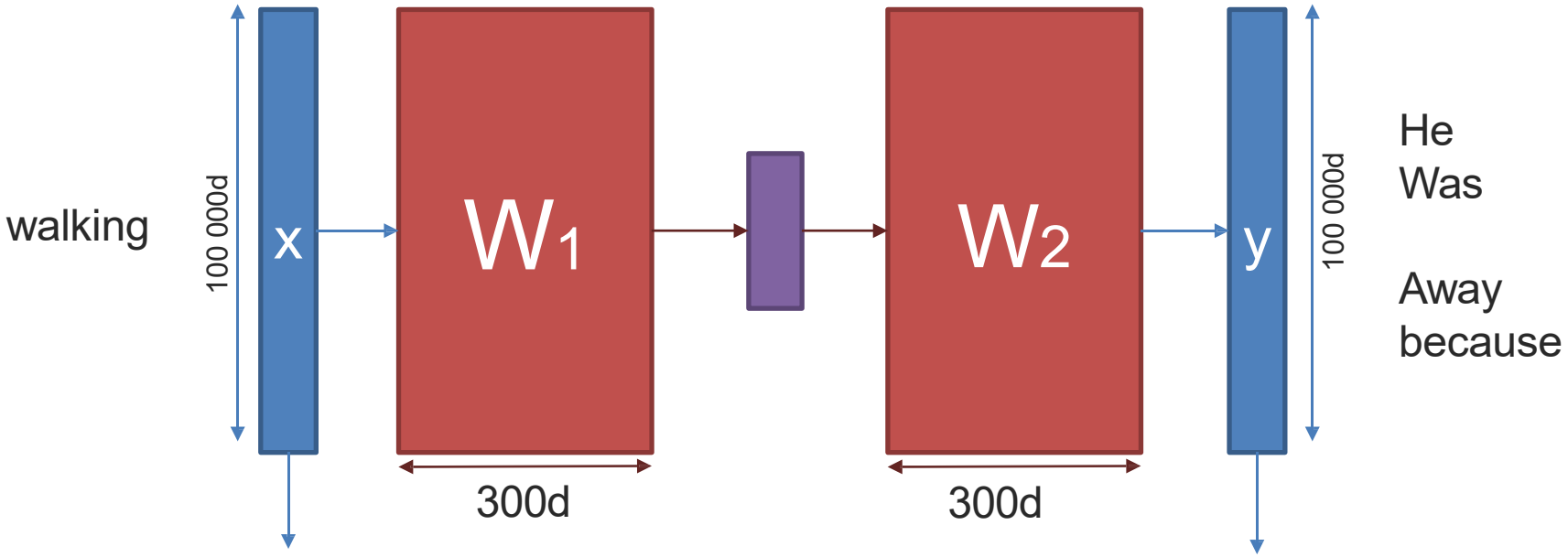
Distribution hypothesis: Approximate the word meaning by its surrounding words

Words used in a similar context will lie close together

Instead of capturing co-occurrence counts directly, predict surrounding words of every word

$$\frac{1}{T}\sum_{t=1}^{T}\sum_{-c\leq j\leq c, j\neq 0}\log p(w_{t+j}|w_t)$$

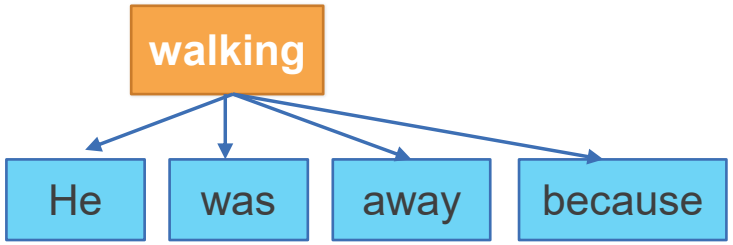
How to learn (word) features/representations?



Word2vec algorithm: https://code.google.com/p/word2vec/

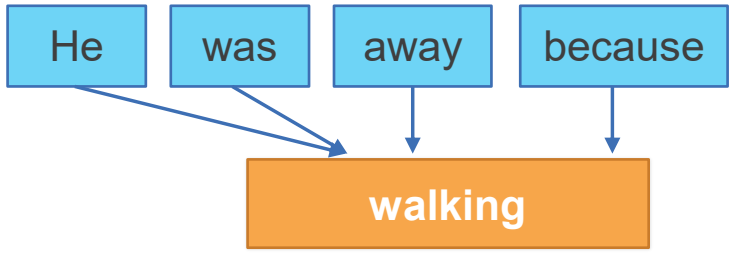
Skip-gram, CBOW, and Negative Sampling

Skip-gram



center word → predict nearby words

CBOW



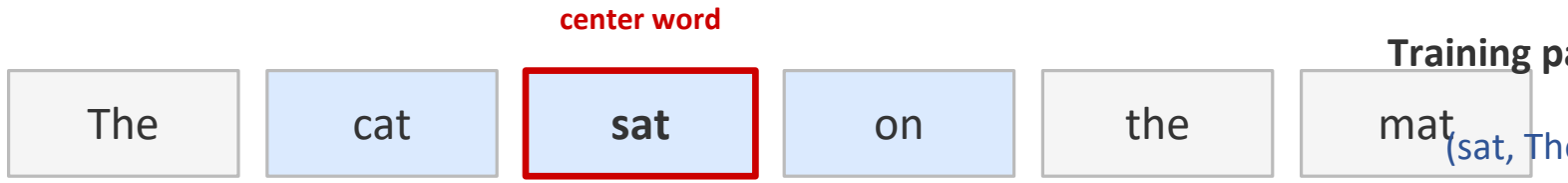
nearby words → predict the center word

Why negative sampling matters

- A full softmax scores all vocabulary words at every update.
- Negative sampling replaces that with 1 positive target plus K sampled “noise” words.
- This makes Word2Vec practical for vocabularies with 100K+ words.
- In practice: Skip-gram often helps rare words; CBOW is faster on frequent words.

train on a few contrasts, not the whole vocabulary

Word2Vec Training: The Sliding Window



Training pairs generated:

(sat, The)

(sat, cat)

(sat, on)

(sat, the)

center → context

How it works: Slide a window across every sentence in the corpus. At each position, the center word is the input and each context word is a separate training target.

Scale: A single pass over 1 billion words with window=5 generates ~10 billion training pairs. This is why Word2Vec learns so much from raw text.

The window slides one word at a time. Every word in the vocabulary gets to be the center word many times, accumulating gradient signal from diverse contexts.

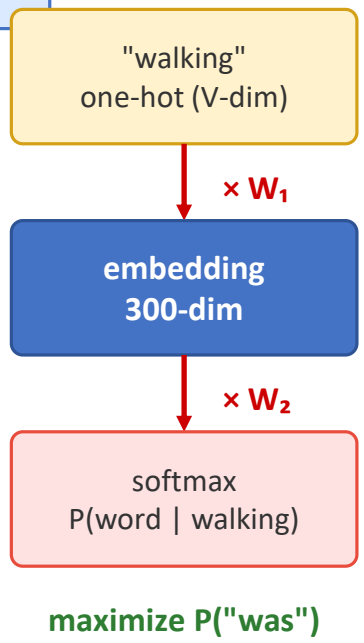
Skip-gram: Predict Context from Center Word

Step 1: Pick center word

Step 2: Generate (center, context) pairs

walking	→	was	*context left-1*
walking	→	He	*context left-2*
walking	→	away	*context right-1*
walking	→	because	*context right-2*

Step 3: Train the network



maximize P("was")

Key insights

One center word generates 2×window training pairs (here: 4 pairs).

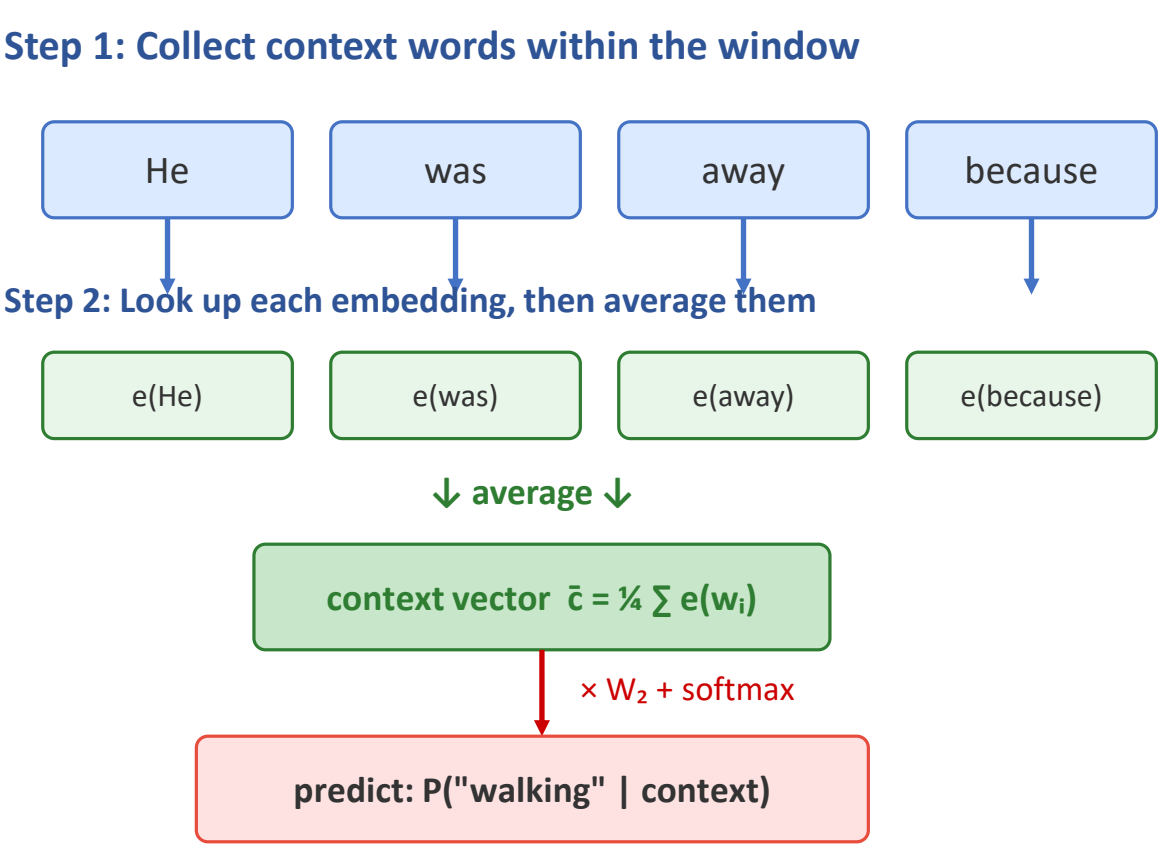
W_1 IS the embedding matrix. Each row = one word's vector.

After training, discard W_2. Keep W_1 as your word embeddings.

Since one-hot × W_1 = row lookup, training is efficient.

Words in similar contexts get similar W_1 rows because they are trained to predict the same context words.

CBOW: Predict Center Word from Context



CBOW vs Skip-gram

CBOW

Input: all context words (averaged)
Output: predict the center word
One prediction per window position
Faster training, smooths over rare words

Skip-gram

Input: the center word alone
Output: predict each context word separately
Multiple predictions per window position
More training signal, better for rare words

Which to use?

Skip-gram is the default choice for most applications. CBOW is preferred when training speed matters more than rare-word quality.

The Softmax Bottleneck Problem

Full Softmax (expensive)

For each training pair, compute score for ALL V words:

the	cat	sat	on
a	mat	dog	ran
was	is	he	she
it	to	of	...

Must score EVERY word in vocabulary

V = 100,000 words → 100K dot products per training pair

Cost: O(V) per example = impossibly slow at scale

Negative Sampling (practical)

Only score 1 positive + K negative samples:

✅ **"was" — real context word (push score UP)**

❌ "cat" — random noise word (push score DOWN)

❌ "pizza" — random noise word (push score DOWN)

❌ "running" — random noise word (push score DOWN)

❌ "blue" — random noise word (push score DOWN)

❌ "graph" — random noise word (push score DOWN)

Only K+1 dot products per training pair

K = 5–15 typically → 6–16 dot products vs 100K

Cost: O(K) per example = 10,000× faster!

Negative Sampling: How It Works

The training objective for one (center, context) pair:

Maximize: $\log \sigma(v'_{was}{}^{T} \cdot v_{walking})$

"Push the dot product of real pairs toward +∞ (sigmoid → 1)"

Plus for each negative sample k = 1..K:

$\log \sigma(-v'_{noise\_k}{}^{T} \cdot v_{walking})$

"Push the dot product of fake pairs toward −∞ (sigmoid → 0)"

How are negative words sampled?

$$P(w) \propto freq(w)$$

The 3/4 power upweights rare words relative to their raw frequency. Without it, common words like 'the' would dominate all negative samples.

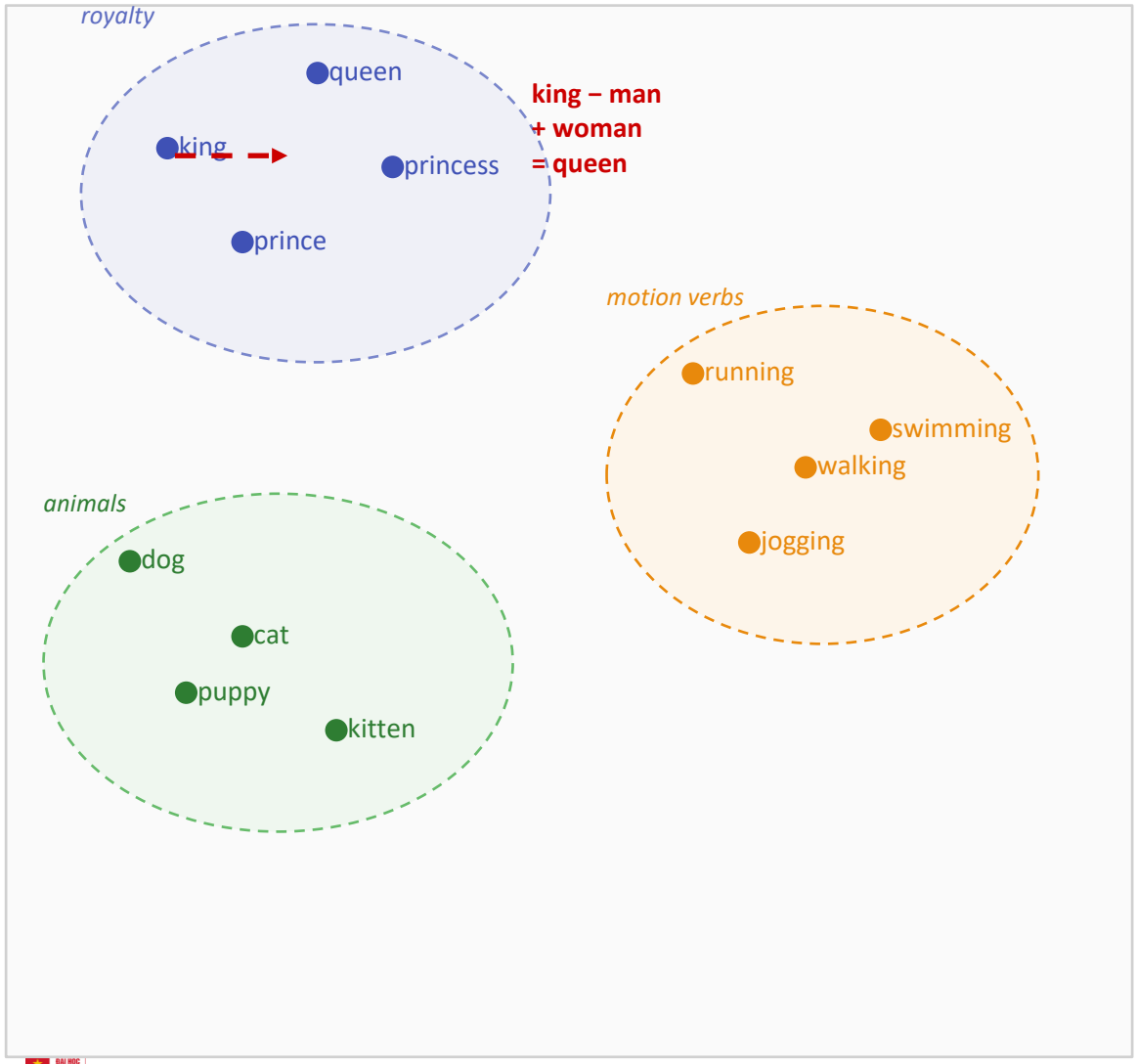
Intuition

Instead of asking: “Of all 100K words, which is most likely?”

We ask: “Is "was" a real context word, or is it noise like "pizza"?”

Binary classification (real vs noise) is much cheaper than V-way classification. And it turns out to produce equally good embeddings!

What Word2Vec Learns: The Embedding Space



What emerges from training

Semantic clusters

Words used in similar contexts cluster together: {king, queen, prince} all appear near words like “royal, throne, crown.”

Linear relationships

Consistent vector offsets encode relationships:

- gender: man → woman ≈ king → queen
- tense: walk → walked ≈ run → ran
- country-capital: France → Paris ≈ Japan → Tokyo

Dimensionality

Typical: 100–300 dimensions. More dims = finer distinctions but slower and more data needed. 300d is the standard.

These properties are not programmed — they emerge automatically from predicting context words at scale.

How to use these word representations

If we would have a vocabulary of 100 000 words:

Classic NLP: 100 000 dimensional vector

Walking: [0; 0; 0; 0;….; 0; 0; 1; 0;…; 0; 0]

Running: [0; 0; 0; 0;….; 0; 0; 0; 0;…; 1; 0]

⇒ Similarity = 0.0

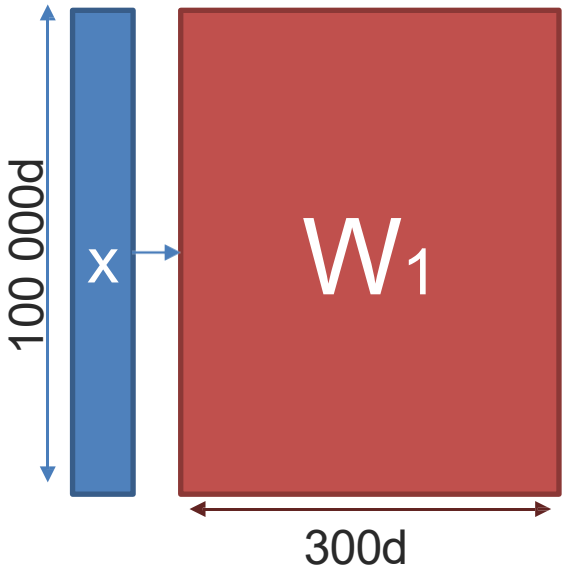
⇓ Transform: x'=x*W

Goal: 300 dimensional vector

Walking: [0,1; 0,0003; 0;….; 0,02; 0.08; 0,05]

Running: [0,1; 0,0004; 0;….; 0,01; 0.09; 0,05]

⇒ Similarity = 0.9



Vector space models of words

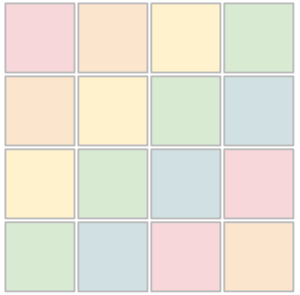
- While learning these word representations, we are actually building a vector space in which all words reside with certain relationships between them
- Encodes both syntactic and semantic relationships
- This vector space allows for algebraic operations:

$$\text{Vec(king)} - \text{vec(man)} + \text{vec(woman)} \approx \text{vec(queen)}$$

GloVe: Global Vectors from Co-occurrence Counts

Prediction-based and count-based methods are closely related.

co-occurrence matrix X_{ij}



weighted least squares objective

$$w_i^T \tilde{w}_j + b_i + \tilde{b}_j \approx \log X_{ij}$$

dense word vectors

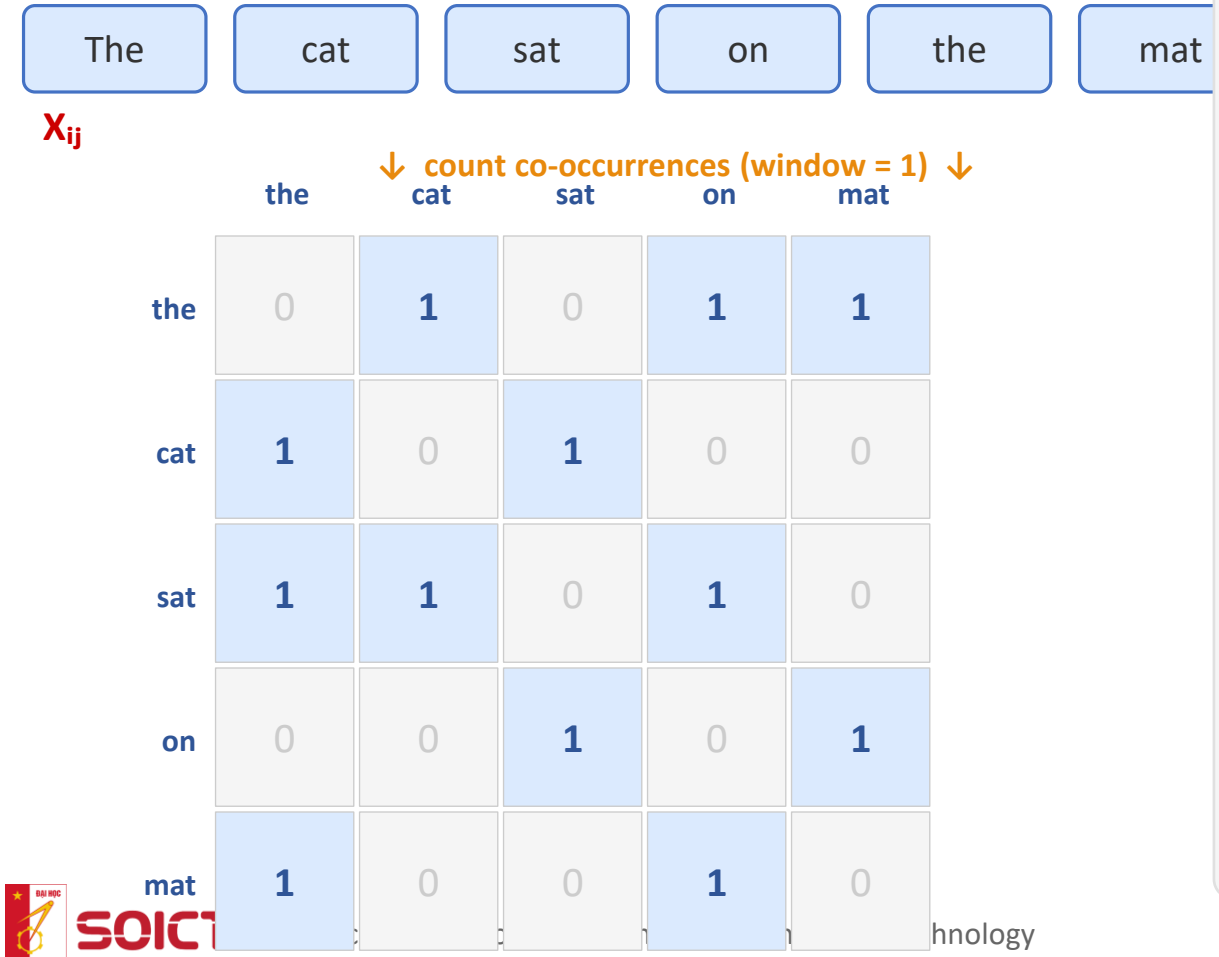
- Word2Vec learns embeddings by predicting nearby words from local windows.
- GloVe starts from the full co-occurrence matrix and learns vectors whose dot products match global count statistics.
- The two families often produce similar geometric structure, but GloVe makes the matrix-factorization view explicit.
- This is why GloVe is often described as a bridge between classical count models and neural prediction models.

global corpus structure → usable static embeddings

Step 1: Build the Co-occurrence Matrix

Scan the entire corpus and count how often each word pair appears within a window.

Example corpus sentence:



What is X_{ij}?

X_{ij} = number of times word j appears in the context of word i across the entire corpus.

Properties of the matrix:

- Symmetric: $X_{ij} = X_{ji}$ (if i is near j, j is near i)
- Sparse: most word pairs never co-occur
- Huge: V × V where V = 100K+
- Dominated by common pairs: (the, of) has enormous counts

Contrast with Word2Vec:

Word2Vec never builds this matrix explicitly. It processes one (center, context) pair at a time from a sliding window.

GloVe builds the full matrix first, then learns vectors from it.

Step 2: Learn Vectors to Match the Counts

GloVe objective: minimize $\sum_{i,j} f(X_{ij}) \cdot (w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$

Learn vectors w_i so that their dot product approximates log co-occurrence counts

Breaking it down:

$w_i^T \cdot \tilde{w}_j + b_i + \tilde{b}_j$

Dot product of word vector w_i and context vector $\tilde{w}_j$, plus bias terms.
This should approximate log X_{ij} after training.

$\log X_{ij}$

The logarithm of the co-occurrence count. Using log compresses the huge range of raw counts (1 to millions) into a manageable scale.
Target for the dot product to match.

$f(X_{ij})$ — the weighting function

f(x) = (x/x_max)^0.75 if x < x_max
f(x) = 1 if x ≥ x_max
Caps the influence of very frequent pairs like (the, of). Without this, common pairs would dominate the loss and distort the embedding space.

$(\dots)^2$ — squared error

The loss for each (i, j) pair is the squared difference between the predicted dot product and the true log count.
This is a regression objective, not classification.
GloVe = weighted least squares on a matrix.

The Deep Connection: GloVe ≈ Word2Vec

Word2Vec (prediction)

Training loop:

for each word in corpus:
  for each context word:
    update embeddings

Processes pairs one at a time
from a sliding window

Online / stochastic training
Never sees the full co-occurrence picture

Implicitly factorizes
a shifted PMI matrix

same result!

GloVe (counting + regression)

Training:

1. Build full X_{ij} matrix (one pass)
2. Fit vectors via weighted least squares

Processes all pairs from
the pre-computed matrix

Batch optimization
Uses global corpus statistics

Explicitly factorizes
the log co-occurrence matrix

Levy & Goldberg (2014): Skip-gram with negative sampling implicitly factorizes a shifted PMI matrix. GloVe makes this factorization explicit. Both arrive at nearly identical embedding spaces.

fastText: Subword-Aware Embeddings

Key idea: represent each word as a bag of character n-grams

"walking" = **< walking >**

<wa | wal | alk | lki | kin | ing | ng>

character 3-grams (with boundary markers < >)

+ the whole word:

walking

Vwalking = Vword + ∑ Vn-gram *(sum of all n-gram vectors + whole-word vector)*

fastText ships pretrained vectors for **157 languages** (fasttext.cc)

Why this is powerful:

Handling unseen words (OOV)

Never seen "walkability" in training?
Its n-grams (wal, alk, abi, bil, ili, lit, ity) overlap with known words!
→ fastText composes a reasonable vector from shared subwords.

Morphological awareness

"walk", "walking", "walked", "walks"
all share n-grams: wal, alk, lk>, <wa

→ Their embeddings are automatically similar!

This is crucial for morphologically rich languages (Finnish, Turkish, Vietnamese compounds) where one root can have dozens of surface forms.

Static Embeddings: One Vector, Many Meanings?

The problem with static embeddings:

"I deposited money in the **bank**."

→ *financial institution*

"The river **bank** was covered in wildflowers."

→ *edge of a river*

Same vector!

vec("bank")
= [0.3, -0.1, ...]

The solution (coming in next lectures):

Contextual embeddings (ELMo, BERT)

"bank" in "deposited money" → vec_1 = [0.8, 0.2, ...]
"bank" in "river bank" → vec_2 = [-0.1, 0.7, ...]

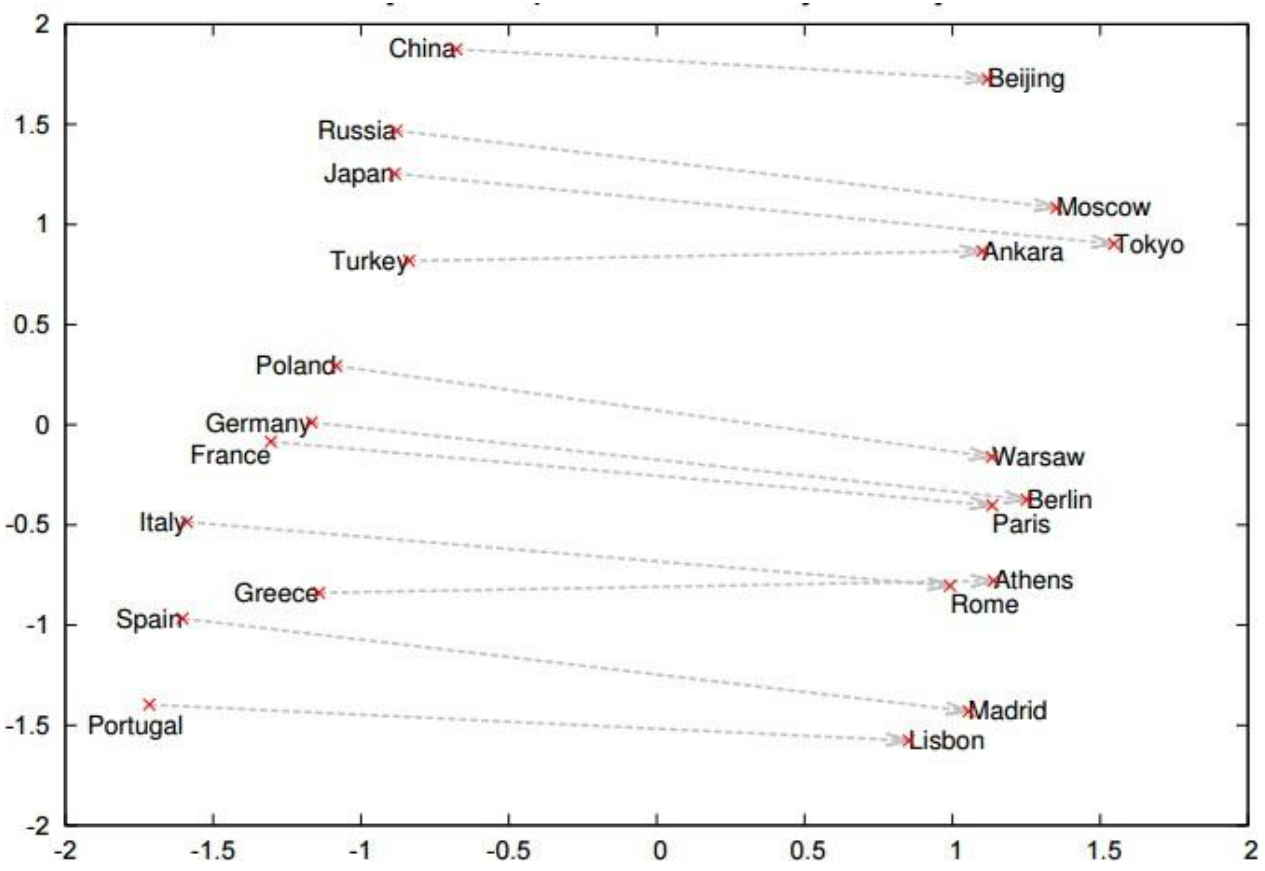
Different vectors for the same word in different contexts!

What we covered in this section:

- Word2Vec: predict context → dense embeddings
- GloVe: factorize co-occurrence counts → same result
- fastText: add subword n-grams → handle OOV + morphology

All three produce static, one-vector-per-word embeddings.
Contextual models fix the polysemy problem but need more compute.

Vector space models of words: semantic relationships



Trained on the Google news corpus with over 300 billion words

Do these work?
Issues of bias here

e.g. https://arxiv.org/abs/1607.06520
https://aclanthology.org/W14-1618.pdf

vec(programmer) – vec(man) +
vec(woman) ≈ vec(homemaker)

Sentence Modeling and Recurrent Networks

Comparing Word Embedding Methods

Method	Approach	Key Idea	Strengths	Limitations
Word2Vec (2013)	Prediction-based (local window)	Predict context words from center word (Skip-gram) or vice versa (CBOW)	Fast training, good rare-word quality, negative sampling scales well	One vector per word type, no polysemy, no subword info
GloVe (2014)	Count-based (global matrix)	Factorize log co-occurrence matrix so dot products match statistics	Uses global corpus structure, theoretically clean, similar quality to Word2Vec	One vector per type, large memory for co-occurrence matrix
fastText (2017)	Prediction-based (subword n-grams)	Represent words as bags of character n-grams, share info across morphological forms	Handles OOV words, great for morphologically rich languages, 157 languages	Still static (one vector per type), larger model size
ELMo (2018)	Contextual (biLSTM LM)	Train deep biLSTM language model, use hidden states as contextualized word vectors	Different vectors for same word in different contexts, layer-wise features	Slow inference (sequential), superseded by Transformers
BERT (2018)	Contextual (Transformer)	Masked language model with bidirectional self-attention over full context	State-of-the-art on many benchmarks, parallelizable, rich representations	Large model (110M+ params), needs fine-tuning, expensive to train

Static embeddings Contextual embeddings

A Limitation of Static Embeddings: One Word, Many Senses

Two token occurrences of the same word

She sat on the river **bank**.

Context cues: water, shore, river

He deposited cash at the **bank**.

Context cues: cash, loan, deposit

What the model stores

Static embedding (Word2Vec / GloVe)

river bank → v_bank
financial bank → v_bank

One word type = one shared vector

Contextual embedding (ELMo / BERT)

river bank → v_bank^river
financial bank → v_bank^finance

Each token occurrence gets its own vector.

Key takeaway

Static embeddings learn one vector per word type. Contextual models produce a different vector for each token occurrence, which helps with polysemy and word sense disambiguation.

Why Bag-of-Words Fails on Ordered Language

Bag-of-words / average embedding view

Sentence A

dog | bites | man → **Dog bites man**

Sentence B

man | bites | dog → **Man bites dog**

avg(e_dog, e_bites, e_man)
= same vector for both sentences

Order-sensitive composition

Sequence model view A

dog → bites → man
→ h_final^A

Sequence model view B

man → bites → dog
→ h_final^B

Key takeaway

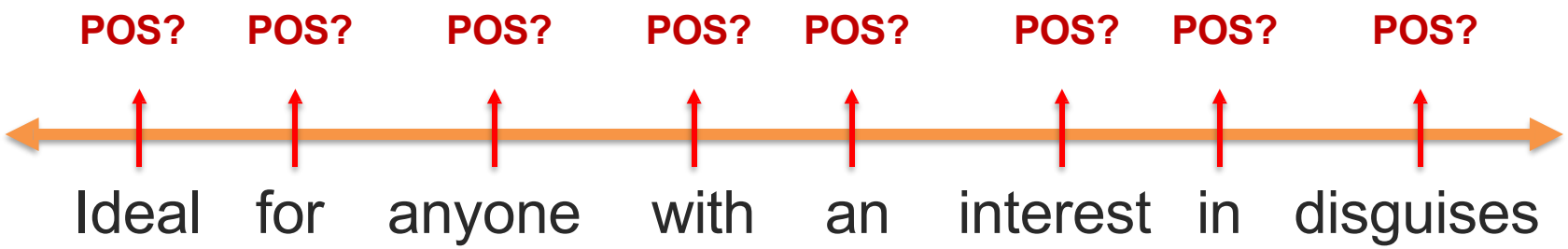
Averaging word vectors throws away order. RNNs, CNNs over text, and attention-based models preserve composition, so “dog bites man” and “man bites dog” no longer collapse to the same representation.

Sentence Modeling: Sequence Prediction

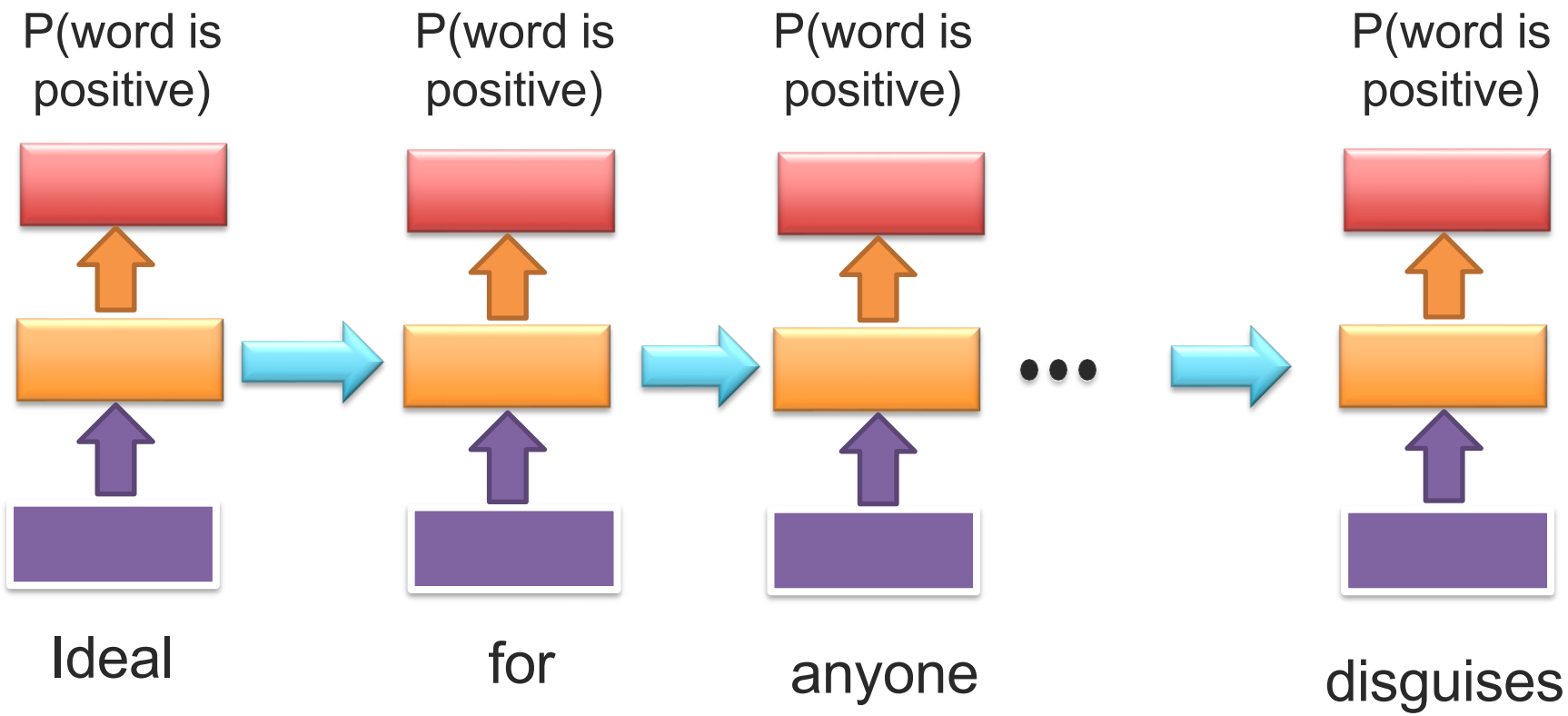
Prediction

Part-of-speech ?
(noun, verb,…)

Sentiment ?
(positive or negative)



RNN for Sequence Prediction

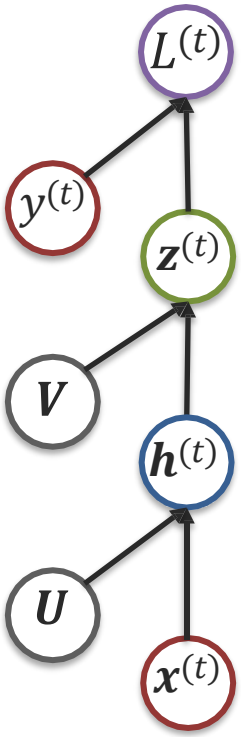


What is the loss?

$$L = \frac{1}{N}\sum_t L^{(t)} = \frac{1}{N}\sum_t -logP(Y = y^{(t)}|z^{(t)})$$

Recurrent Neural Network

Feedforward Neural Network



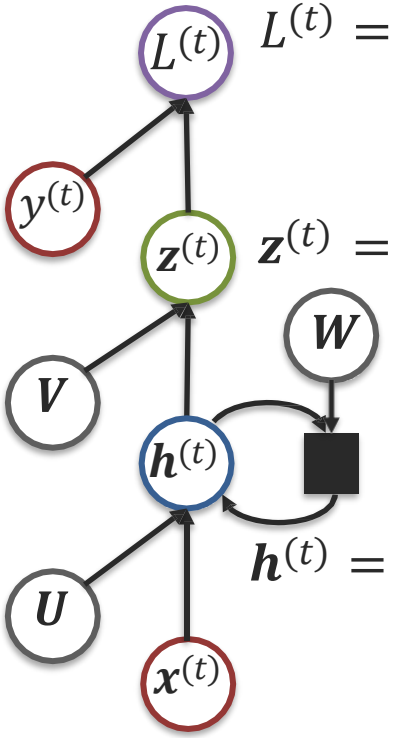
$$L^{(t)} = -logP(Y = y^{(t)}|\boldsymbol{z}^{(t)})$$

$$\boldsymbol{z}^{(t)} = matmult(\boldsymbol{h}^{(t)}, \boldsymbol{V})$$

$$\boldsymbol{h}^{(t)} = tanh(\boldsymbol{U}\boldsymbol{x}^{(t)})$$

Recurrent Neural Networks

$$L = \sum_t L^{(t)}$$



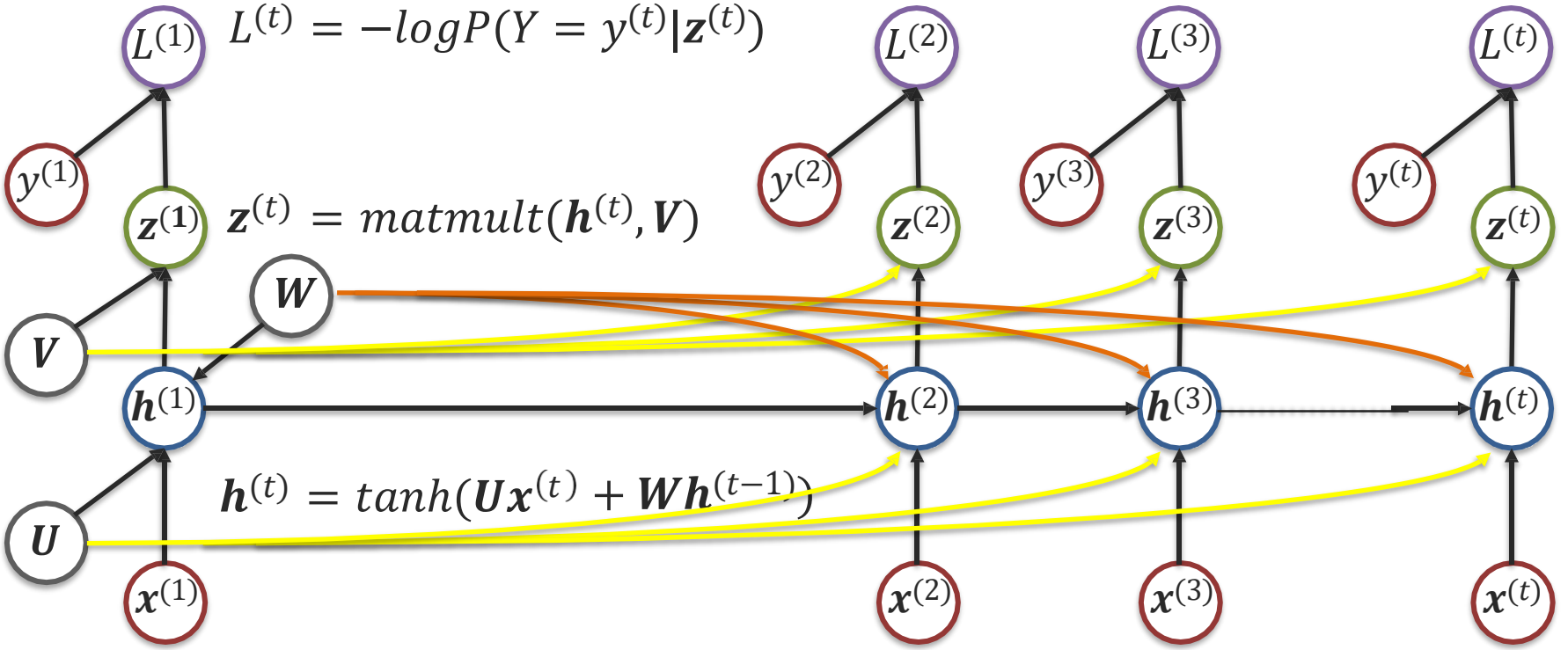
$$L^{(t)} = -logP(Y = y^{(t)}|\boldsymbol{z}^{(t)})$$

$$\boldsymbol{z}^{(t)} = matmult(\boldsymbol{h}^{(t)}, \boldsymbol{V})$$

$$\boldsymbol{h}^{(t)} = tanh(\boldsymbol{U}\boldsymbol{x}^{(t)} + \boldsymbol{W}\boldsymbol{h}^{(t-1)})$$

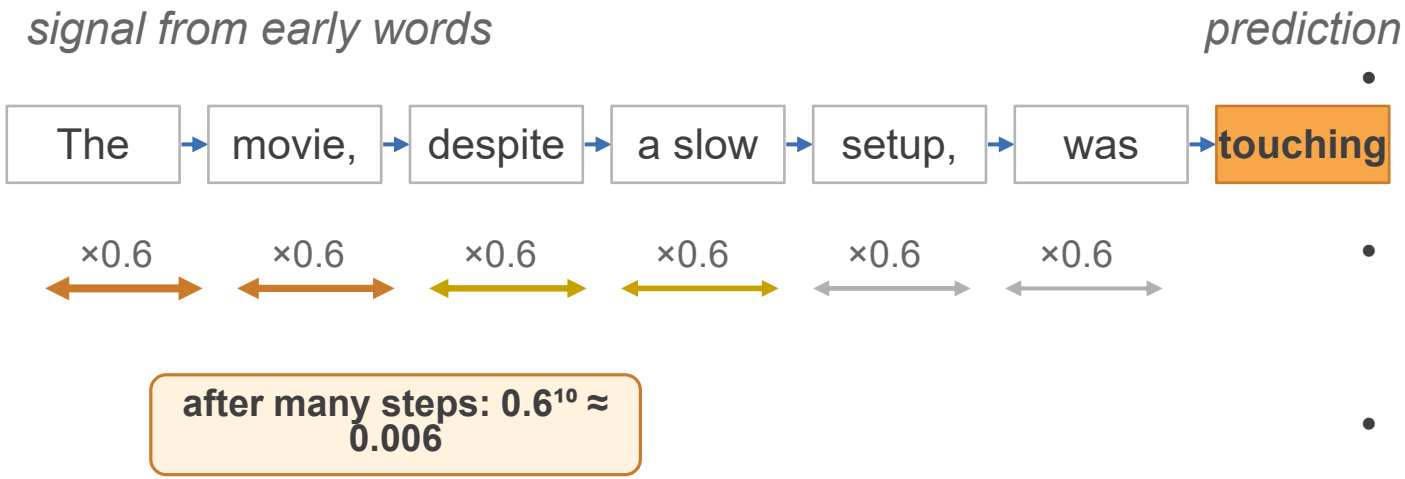
Recurrent Neural Networks - Unrolling

$$L = \sum_t L^{(t)}$$



Same model parameters are used for all time parts.

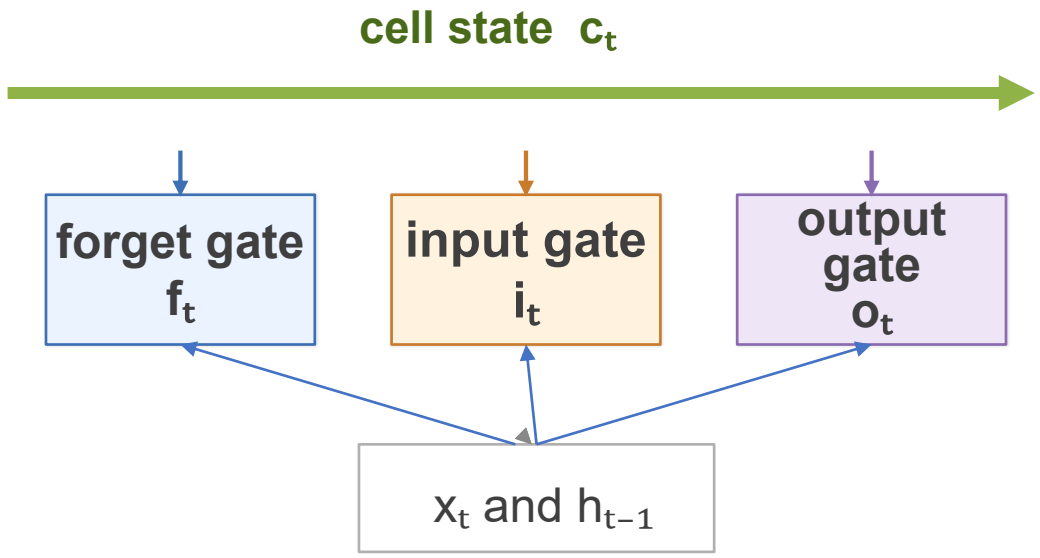
Why Vanilla RNNs Struggle with Long Dependencies



- Backpropagation Through Time multiplies the recurrent Jacobian at every time step.
- If those derivatives are usually smaller than 1, gradients shrink exponentially and early words barely affect learning.
- If they are larger than 1, gradients can explode instead.
- Vanishing/exploding gradients make vanilla RNNs unreliable on long sequences.

gates create a safer path for information and gradients

LSTM: A Memory Cell with Gates



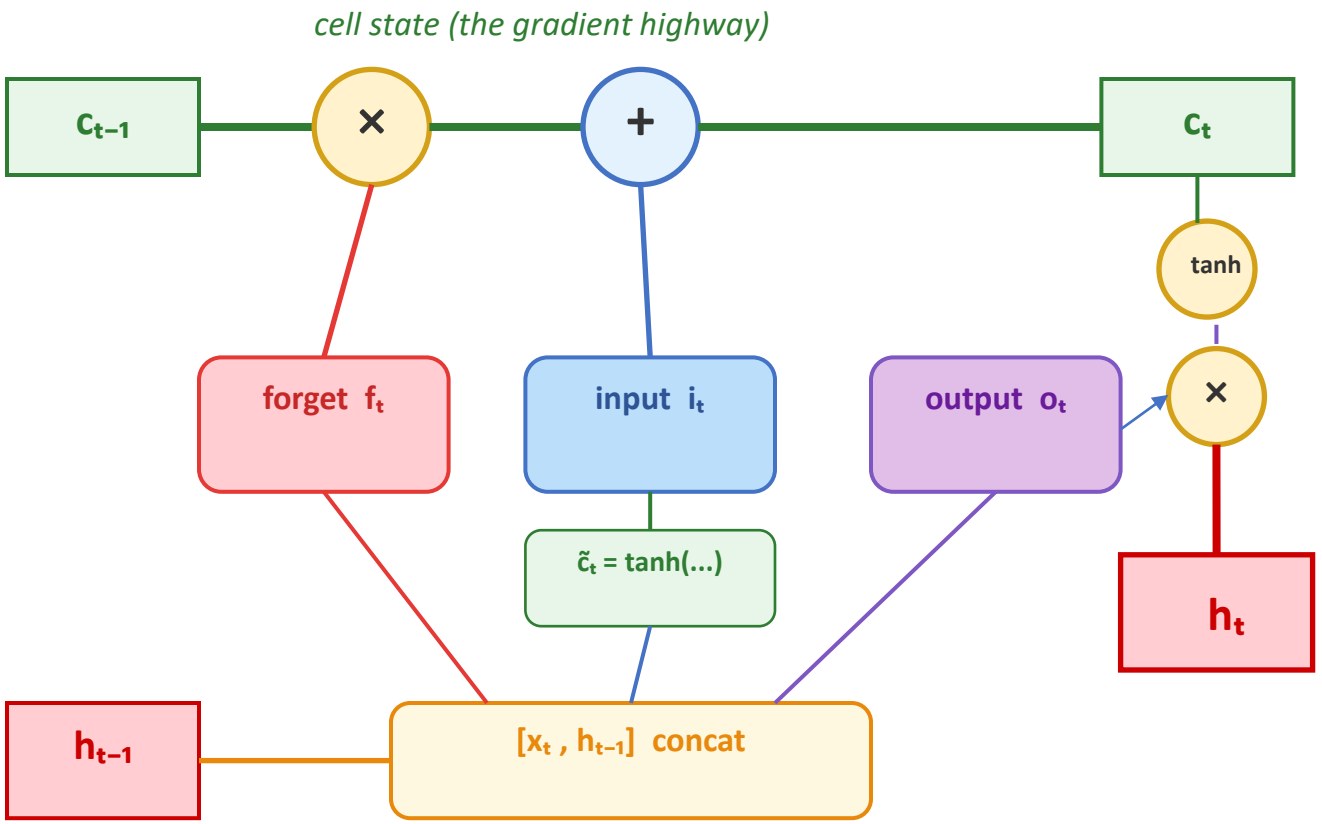
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$h_t = o_t \odot \tanh(c_t)$$

- Forget gate: decide what old memory to keep.
- Input gate: decide what new content to write.
- Output gate: decide what part of memory becomes the visible hidden state.
- The cell state creates a nearly linear “highway,” so gradients can travel farther.

LSTM was the standard fix for long-range sequence modeling before Transformers.

LSTM: Correct Data Flow Through the Cell



Step-by-step computation:

1. Forget gate

$f_t = \sigma(W^f[h_{t-1}, x_t] + b^f)$

What to erase? (0=erase, 1=keep)

2. Input gate

$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$

How much new content to write?

3. Candidate

$\tilde{c}_t = \tanh(W^c[h_{t-1}, x_t] + b^c)$

What new content to add?

4. Update cell

$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$

Forget old + write new

5. Output gate

$o_t = \sigma(W^o[h_{t-1}, x_t] + b^o)$

What to reveal?

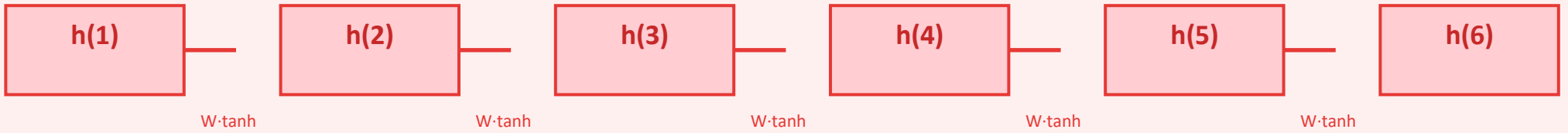
6. Hidden state

$h_t = o_t \odot \tanh(c_t)$

The LSTM output

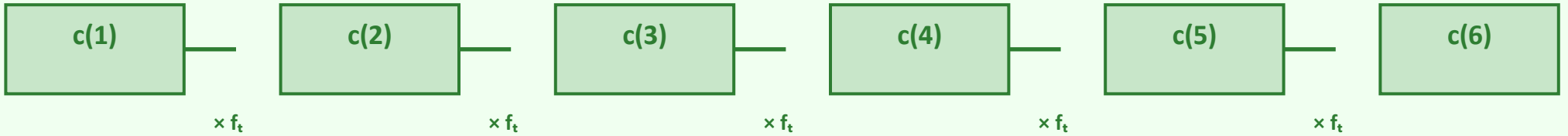
Why LSTM Fixes the Vanishing Gradient Problem

Vanilla RNN: gradient through tanh + W



∂h(t)/∂h(t-1) involves W and tanh' at every step. Each factor < 1 → gradient vanishes exponentially.

LSTM: gradient through cell state



$\partial c(t)/\partial c(t-1) = f_t$ (just the forget gate value!)

f_t can be close to 1 → gradient factor ≈ 1. No matrix multiplication, no tanh squashing. Gradient flows freely through the cell state!

The cell state is a "gradient highway": information and gradients travel many steps without vanishing. This is the core insight of LSTM.

LSTM Gate Intuition: The Notebook Analogy

Think of the cell state as a notebook, and the gates as actions at each step:

Forget Gate f_t

"Erase old notes"

Read current input x_t
and previous output h_{t-1}.

Decide which parts of the
notebook to erase.

$f_t = 0 \rightarrow$ erase completely
$f_t = 1 \rightarrow$ keep everything

Example: seeing "." might
erase the previous subject.

Input Gate i_t

"Write new notes"

Create candidate content
$\tilde{c}_t = \tanh(W[h, x])$.

Decide how much of it
to actually write.

$i_t = 0 \rightarrow$ write nothing
$i_t = 1 \rightarrow$ write fully

Example: "Paris" after
"traveled to" writes
the new location.

Output Gate o_t

"Read from notebook"

The notebook now has
updated content c_t.

Decide which parts to
read out as the visible h_t.

$o_t = 0 \rightarrow$ keep private
$o_t = 1 \rightarrow$ reveal fully

Example: when predicting
the next word, only topic
info may be needed.

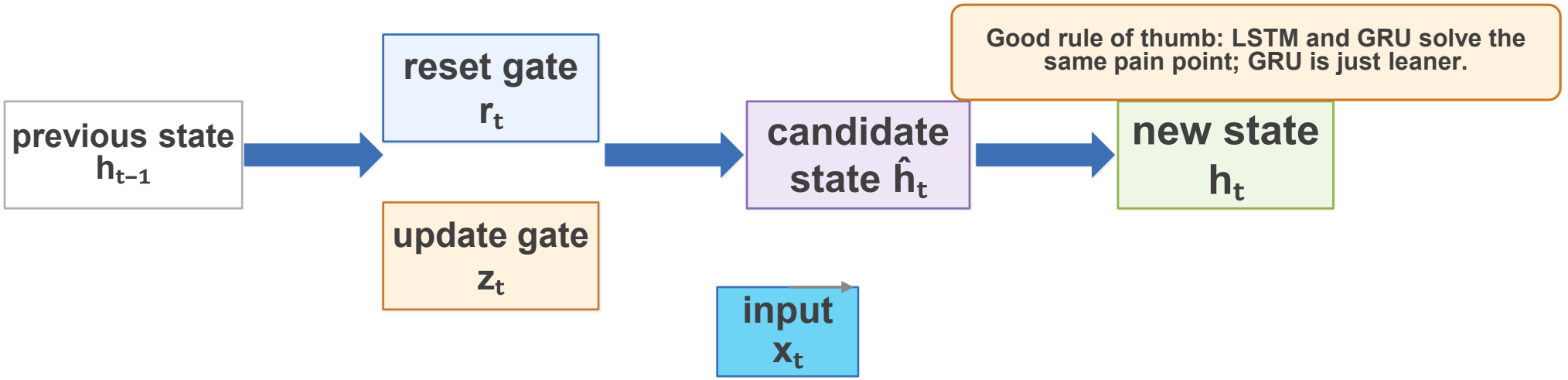
LSTM Walkthrough: Processing a Sentence

"The cat sat on the mat" — tracking the subject through time

Word	f_t	i_t	Cell update	What happens
The	1.0	0.2	start	Low input; nothing meaningful yet
cat	0.9	0.9	+cat	Write 'cat' as subject into memory
sat	0.95	0.7	+sat	Keep 'cat', add action 'sat'
on	0.9	0.3	~same	Keep memory, minor update
the	0.85	0.2	~same	Functional word, small change
mat	0.8	0.8	+mat	Write 'mat' as location

Key observation: f_t stays close to 1 throughout — the LSTM preserves "cat" (the subject) in memory across all 6 steps. A vanilla RNN would have largely forgotten "cat" by the time it reaches "mat". This is why LSTM handles long-range dependencies.

GRU: A Simpler Gated Recurrent Unit



$$\tilde{h}_t = \tanh\left(Wx_t + U(r_t \odot h_{t-1})\right)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

- GRU merges the cell state and hidden state into one vector.
- Reset gate controls how much past information to use when building the candidate state.
- Update gate interpolates between the old state and the new candidate.
- Fewer parameters than LSTM, often similar performance in practice.

GRU

A Simpler Gated Recurrent Unit

Cho et al., 2014 — "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation"

Why Simplify LSTM?

LSTM Complexity

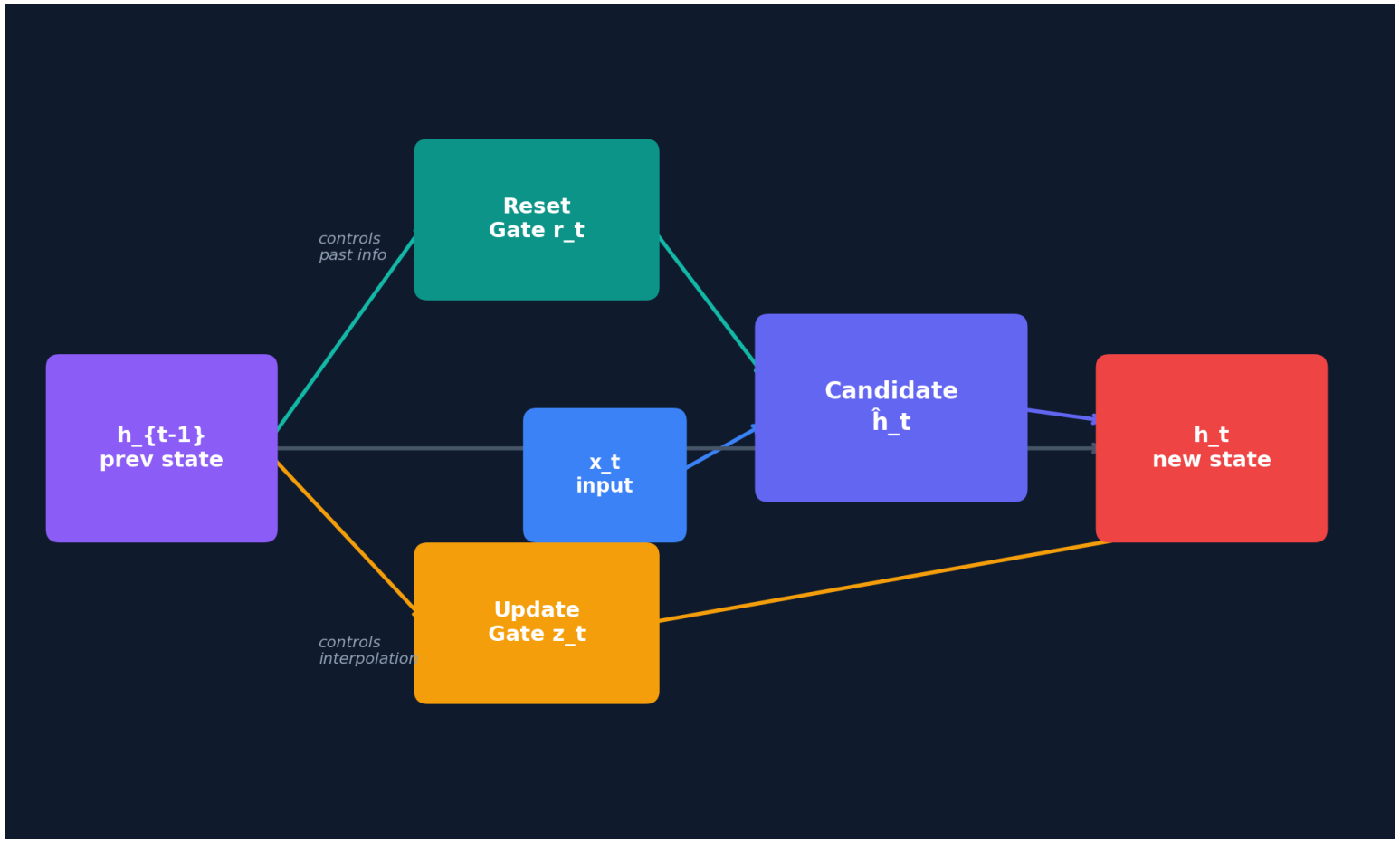
- 3 gates (forget, input, output)
- 2 separate states (cell c_t + hidden h_t)
- 8 weight matrices to learn
- More parameters = slower training

GRU Solution

- Only 2 gates (reset + update)
- Single hidden state h_t merges both roles
- 6 weight matrices → fewer params
- Similar performance in most tasks

Rule of thumb: LSTM and GRU solve the same problem — GRU is just leaner.

GRU Architecture Overview



Data Flow

① x_t and h_{t-1} enter

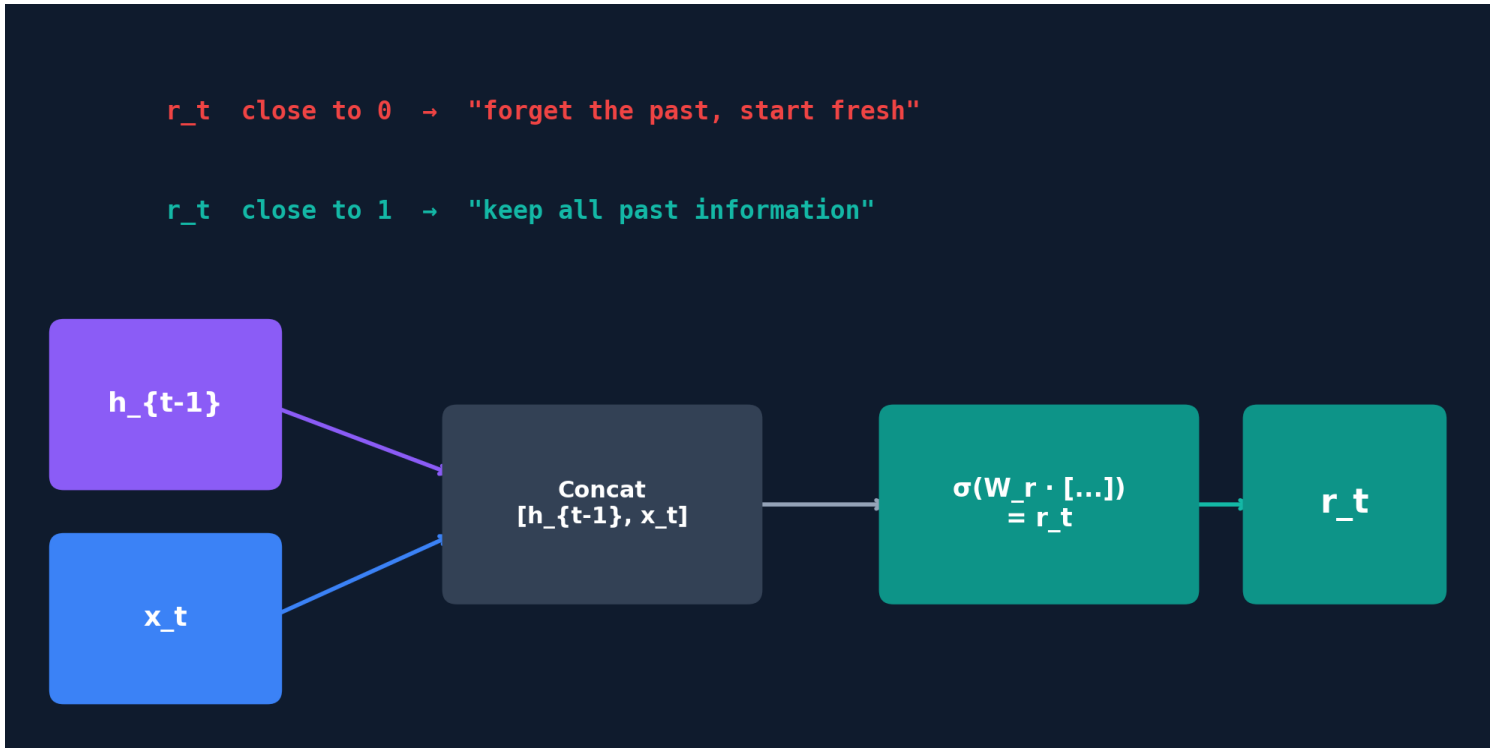
② Reset gate filters h_{t-1}

③ Candidate ĥ_t is computed

④ Update gate interpolates

⑤ New state h_t is output

Step 1: Reset Gate



Formula

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

Output: values in [0, 1]

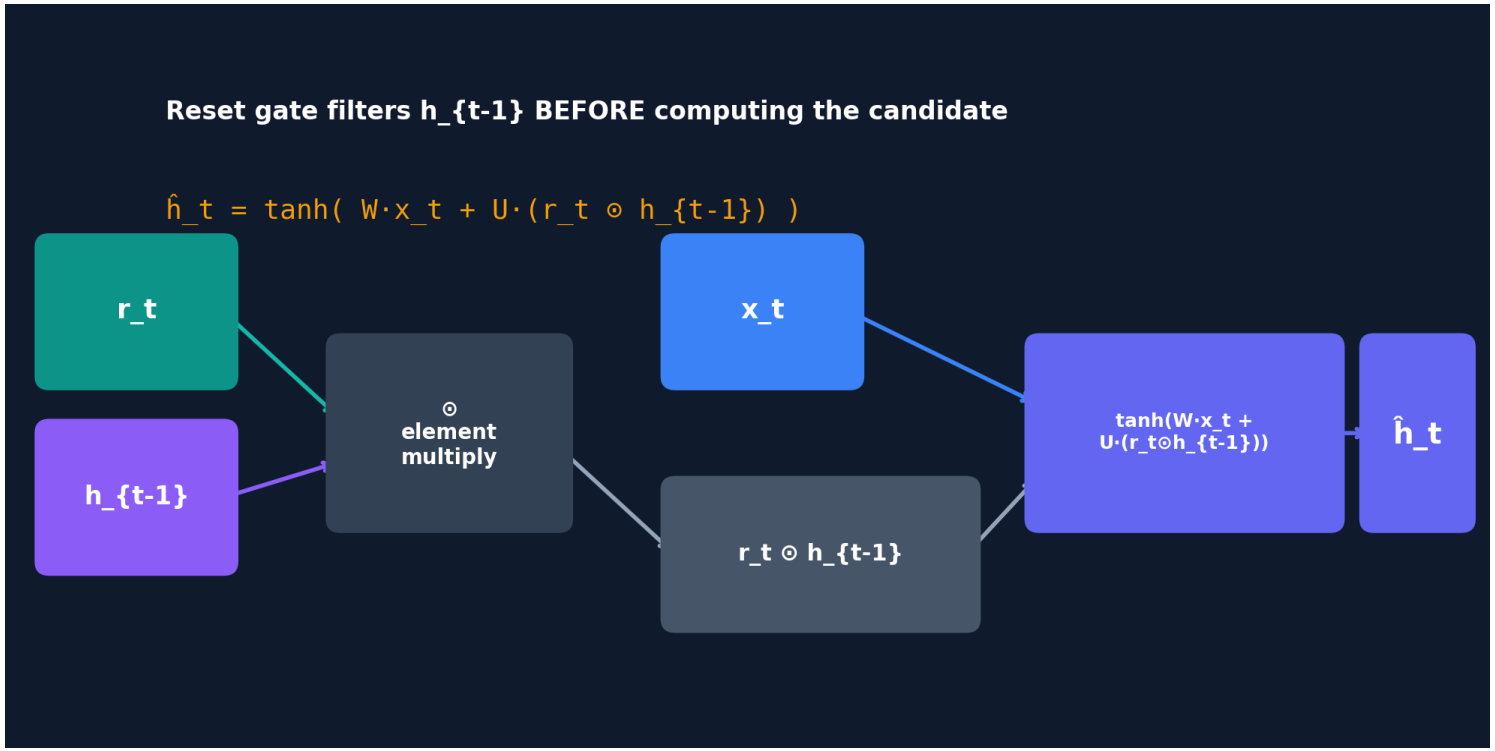
Intuition

$r_t \approx 0$: ignore past, start fresh

$r_t \approx 1$: use full past history

Each dimension can be different — selective memory

Step 2: Candidate State



Formula

$$\hat{h}_t = \tanh(W \cdot x_t + U \cdot (r_t \odot h_{t-1}))$$

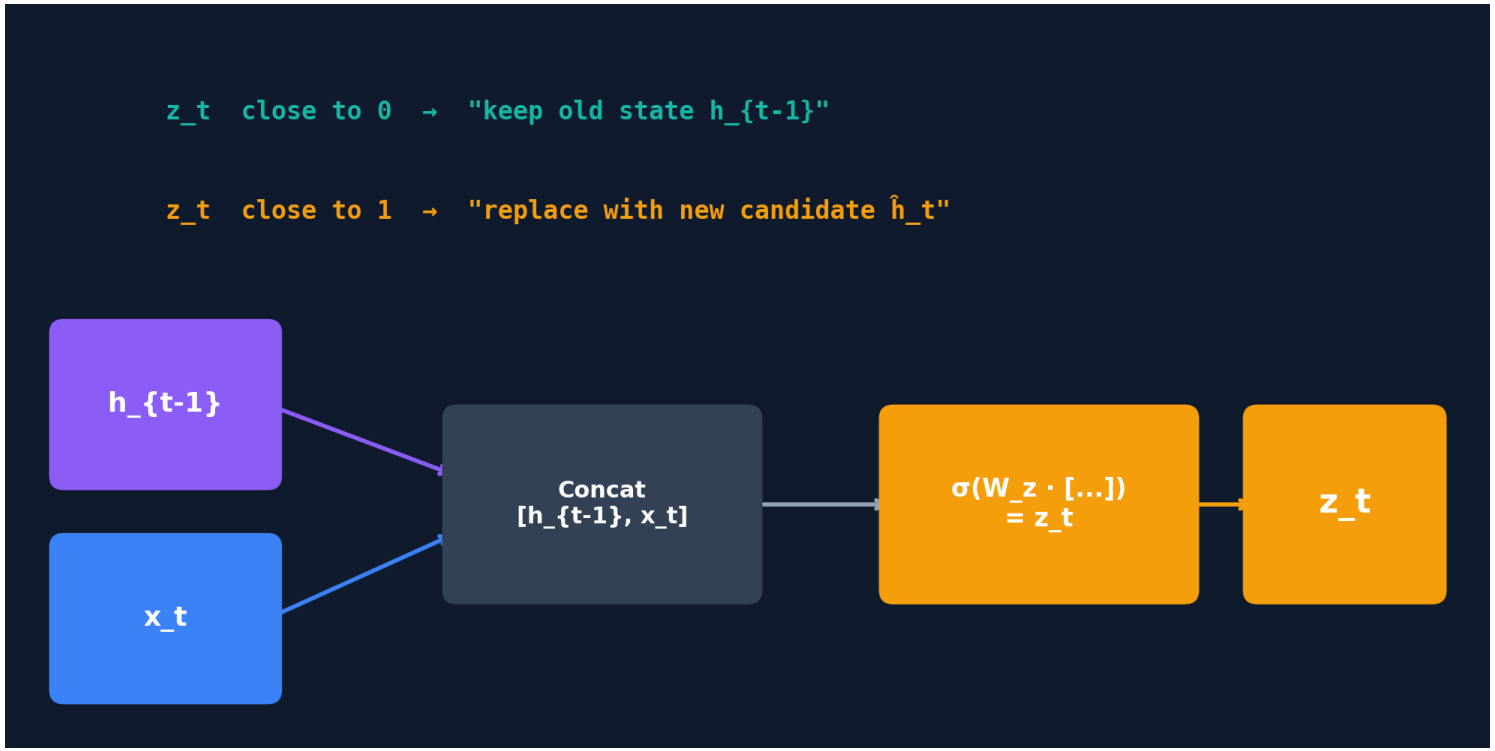
Output: values in [-1, 1]

Key Insight

$r_t \odot h_{t-1}$ is the filtered past

Reset gate acts BEFORE the candidate — this is where GRU decides which memories to bring forward

Step 3: Update Gate



Formula

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

Output: values in [0, 1]

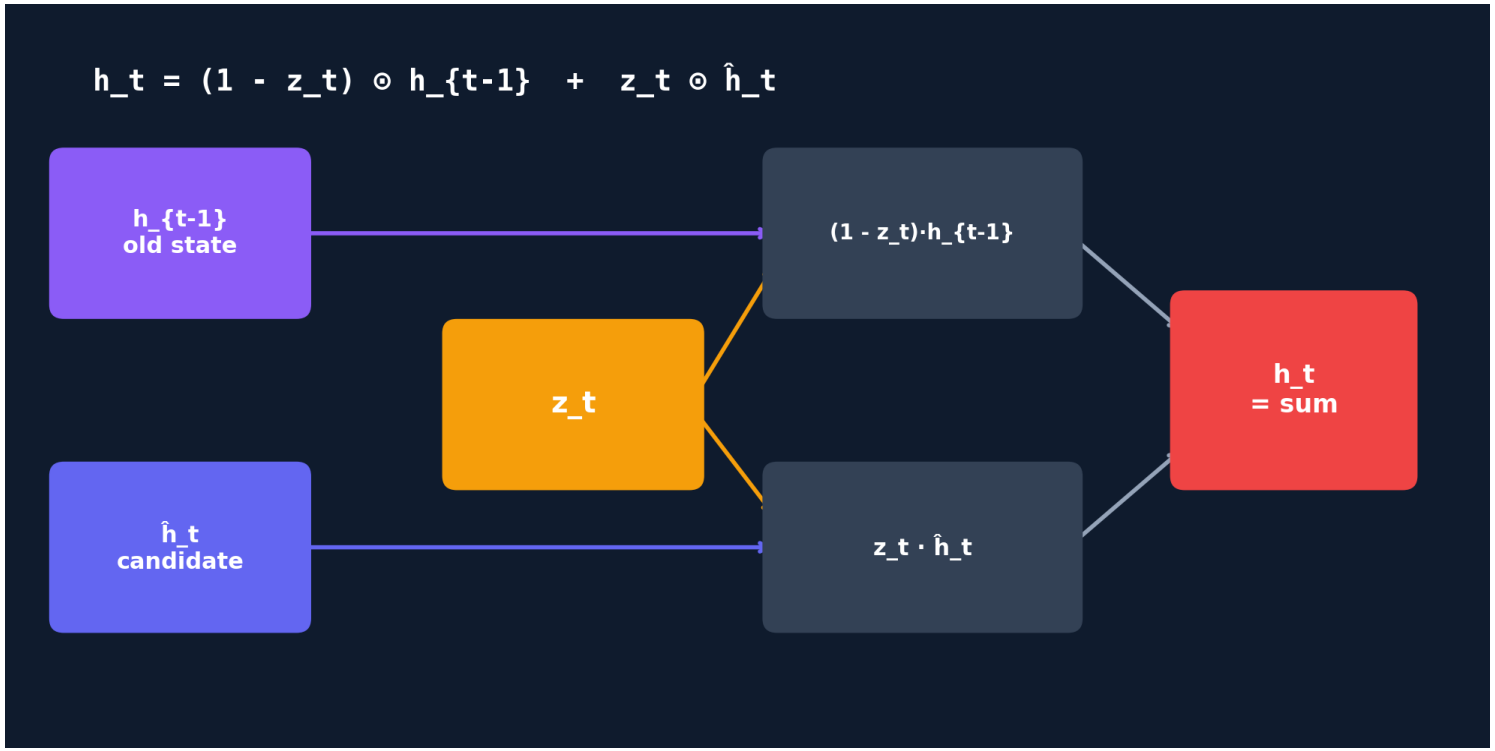
Intuition

$z_t \approx 0$: copy old state (skip input)

$z_t \approx 1$: fully replace with candidate

Combines LSTM's forget + input gates into one!

Step 4: Interpolation → New State



Final Equation

$$h_t = (1-z_t) \odot h_{t-1} + z_t \odot \hat{h}_t$$

Why Interpolation Works

Weights (1-z_t) and z_t always sum to 1

Creates a smooth blend between old and new

When z_t = 0: gradient flows directly through h_{t-1} → solves vanishing gradient!

GRU Walkthrough: Processing a Sentence

"The cat sat on the mat" — tracking the subject through time

Word	r_t (reset)	z_t (update)	State	What happens
The	0.9	0.1	start	Low z_t → keep old state (empty); minor initialization
cat	0.8	0.8	+cat	High z_t → replace ~80% of state with candidate containing 'cat'
sat	0.9	0.6	+sat	r_t high → candidate uses 'cat'; z_t moderate → blend in 'sat'
on	0.7	0.15	~same	Low z_t → state barely changes; 'cat sat' preserved in memory
the	0.6	0.1	~same	Function word, very low z_t → skip, keep 'cat sat' memory intact
mat	0.5	0.7	+mat	r_t moderate → partial reset for new context; z_t high → write 'mat'

Key observation: z_t stays low (0.1–0.15) for function words "on", "the" — the GRU preserves "cat" (the subject) across these steps with minimal state change. **Compare with LSTM:** LSTM uses high f_t ≈ 1 to remember; GRU uses low z_t ≈ 0 to remember. Same effect, opposite convention!

GRU vs. LSTM Comparison

	LSTM	GRU
Gates	3 (forget, input, output)	2 (reset, update)
States	2 (cell c_t + hidden h_t)	1 (hidden h_t only)
Parameters	~4n(n+m)	~3n(n+m) (~25% fewer)
Speed	Slower	Faster
Performance	Slight edge on long-memory tasks	Very similar overall

Summary & Practical Advice

GRU Equations

- $r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$
- $\hat{h}_t = \tanh(W \cdot x_t + U \cdot (r_t \odot h_{t-1}))$
- $z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$
- $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \hat{h}_t$

When to Use

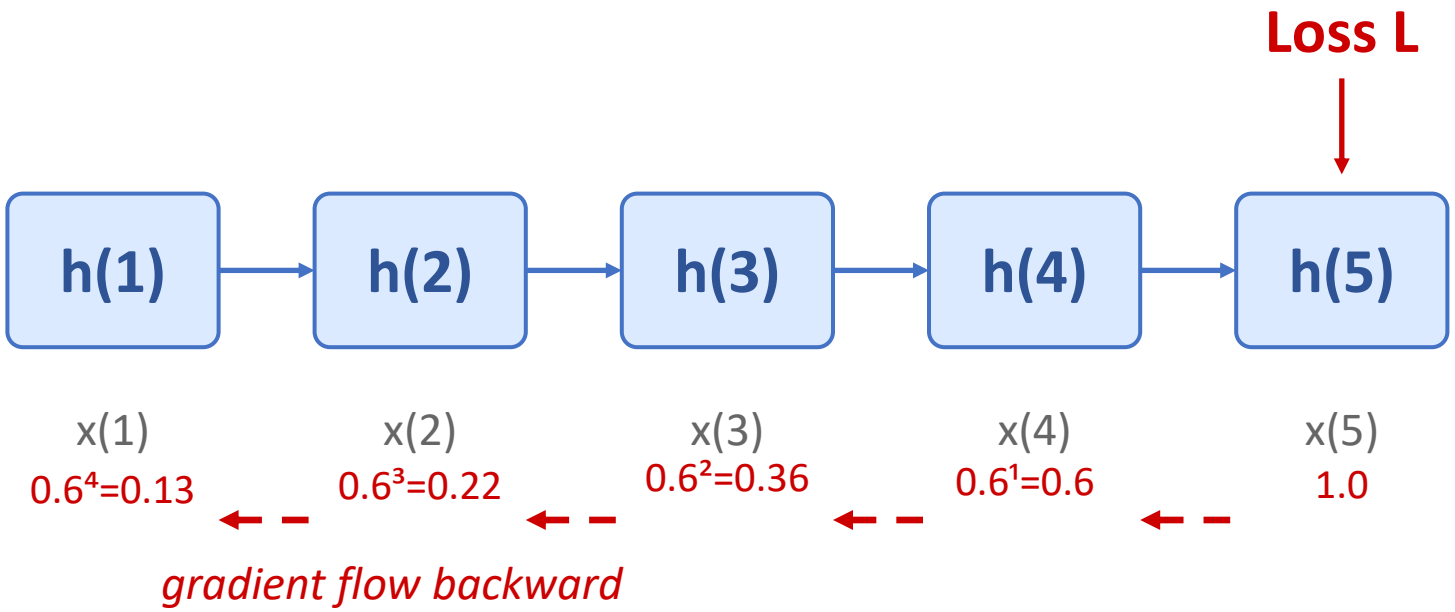
- Audio & speech processing
- Time-series forecasting
- Resource-constrained settings
- Smaller datasets (fewer params)

When NOT to Use

- Most NLP tasks → use Transformers
- Very long sequences (>1000 steps)
- Tasks needing parallelism
- Large-scale language models

"Try both GRU and LSTM, pick whichever works better on your validation set."

Backpropagation Through Time (BPTT)



Forward pass

Compute h(1) → h(2) → ... → h(N) → Loss

Backward pass

Gradients flow backward through the chain. At each step, multiply by ∂h(t)/∂h(t-1).

The problem

If each Jacobian factor < 1, gradient shrinks exponentially.
If > 1, gradient explodes.

Gradient at step 1 =
$\partial L/\partial h(1) = \prod \partial h(t)/\partial h(t-1)$

This product of many terms < 1 vanishes, or > 1 explodes.

Practical RNN Training Tips

1. Padding and masks

- Batch sentences to the same length for efficiency.
- Use masks so PAD tokens do not affect the loss, pooling, or attention.
- For sequence labels, h(N) must refer to the last real token, not the last pad.

2. Teacher forcing

- During training, feed the gold previous token to the decoder.
- This speeds optimization and keeps targets aligned.
- At test time the model consumes its own predictions, creating exposure bias.

3. Clip gradients and truncate BPTT

- Clip the gradient norm to prevent rare exploding updates.
- Backpropagate only K time steps when sequences are very long.
- Truncation saves memory, but it also shortens the learning horizon.

4. Regularize and monitor

- Apply dropout on embeddings, outputs, or between stacked recurrent layers.
- Early stopping and validation curves matter because recurrent models can overfit quickly.
- Always inspect a few decoded predictions, not just scalar metrics.

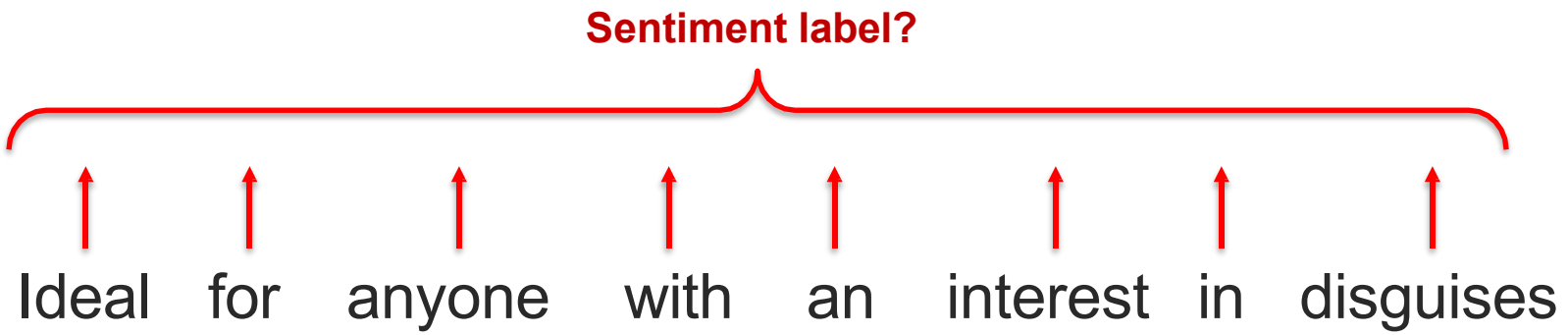
Common recipe

Embedding layer → LSTM/GRU → dropout → linear projection → cross-entropy loss, optimized with Adam plus gradient clipping.

Sentence Modeling: Sequence Label Prediction

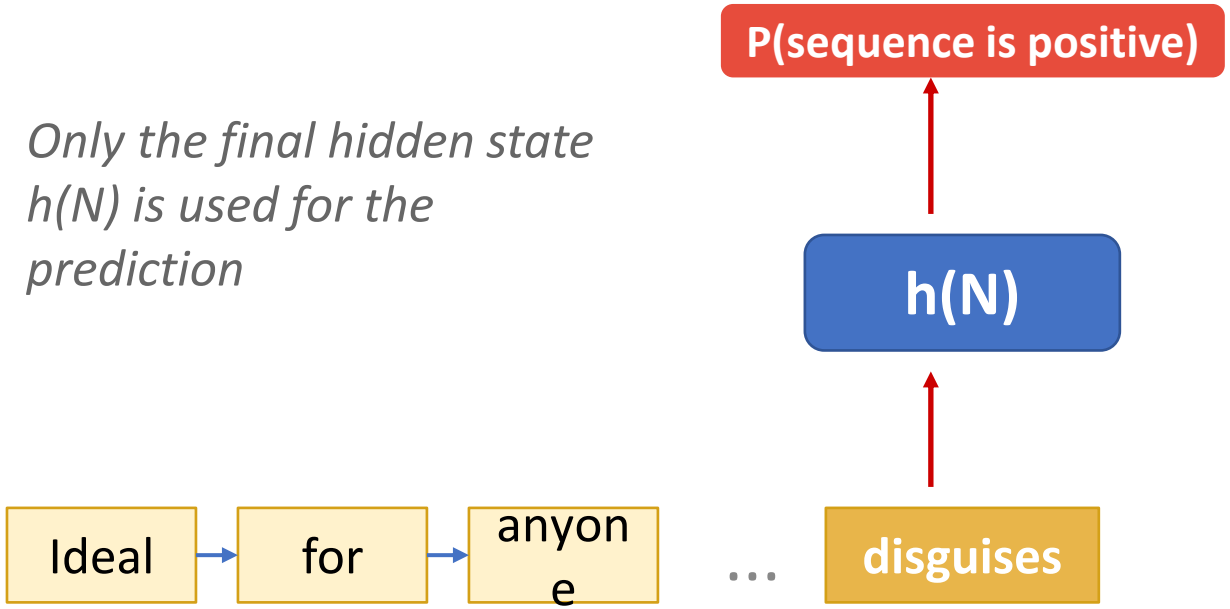
Prediction

Sentiment ?
(positive or negative)



RNN for Sequence Prediction

Only the final hidden state h(N) is used for the prediction



What is the loss?

$$L = L(N) = -\log P(Y = y(N) \mid z(N))$$

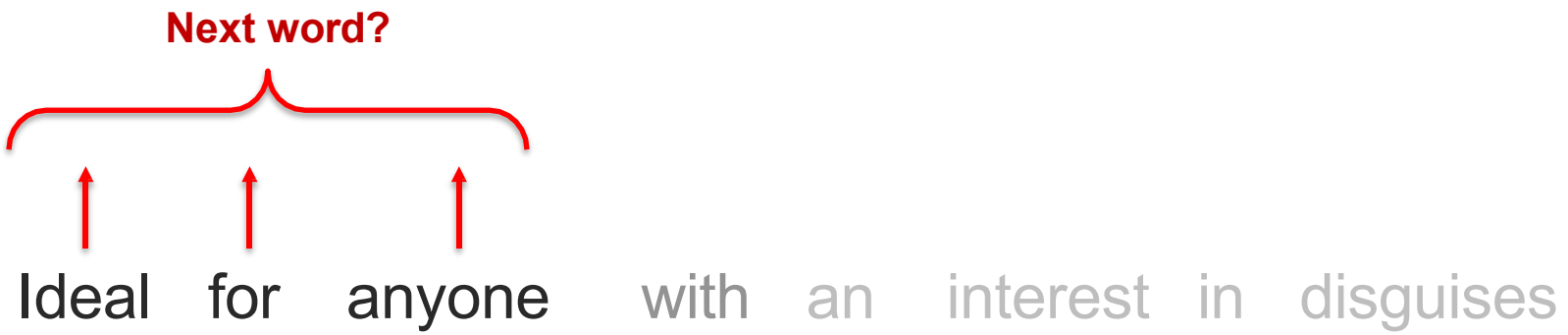
Loss is computed only at the final step, not at every word position.

- The RNN reads the entire sequence, building up context in its hidden states.
- Only the last hidden state h(N) is passed to the output layer.
- This compresses the whole sentence into one fixed-size vector.
- Limitation: information from early words may be lost — motivates LSTM, attention.

Language Models

Sentence Modeling: Language Model

Prediction → Next word



LSTM vs GRU: Side-by-Side Comparison

LSTM

Forget gate f_t — What old memory to erase

Input gate i_t — What new content to write

Output gate o_t — What to expose as h_t

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$
$$h_t = o_t \odot \tanh(c_t)$$

3 gates + separate cell state

GRU

Reset gate r_t — How much past to use for candidate

Update gate z_t — Interpolate old and new state

$$h_t = (1-z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

2 gates, no separate cell state

Fewer parameters → faster training
Similar performance to LSTM in practice
Good default when compute is limited

Language Model Application: Speech Recognition

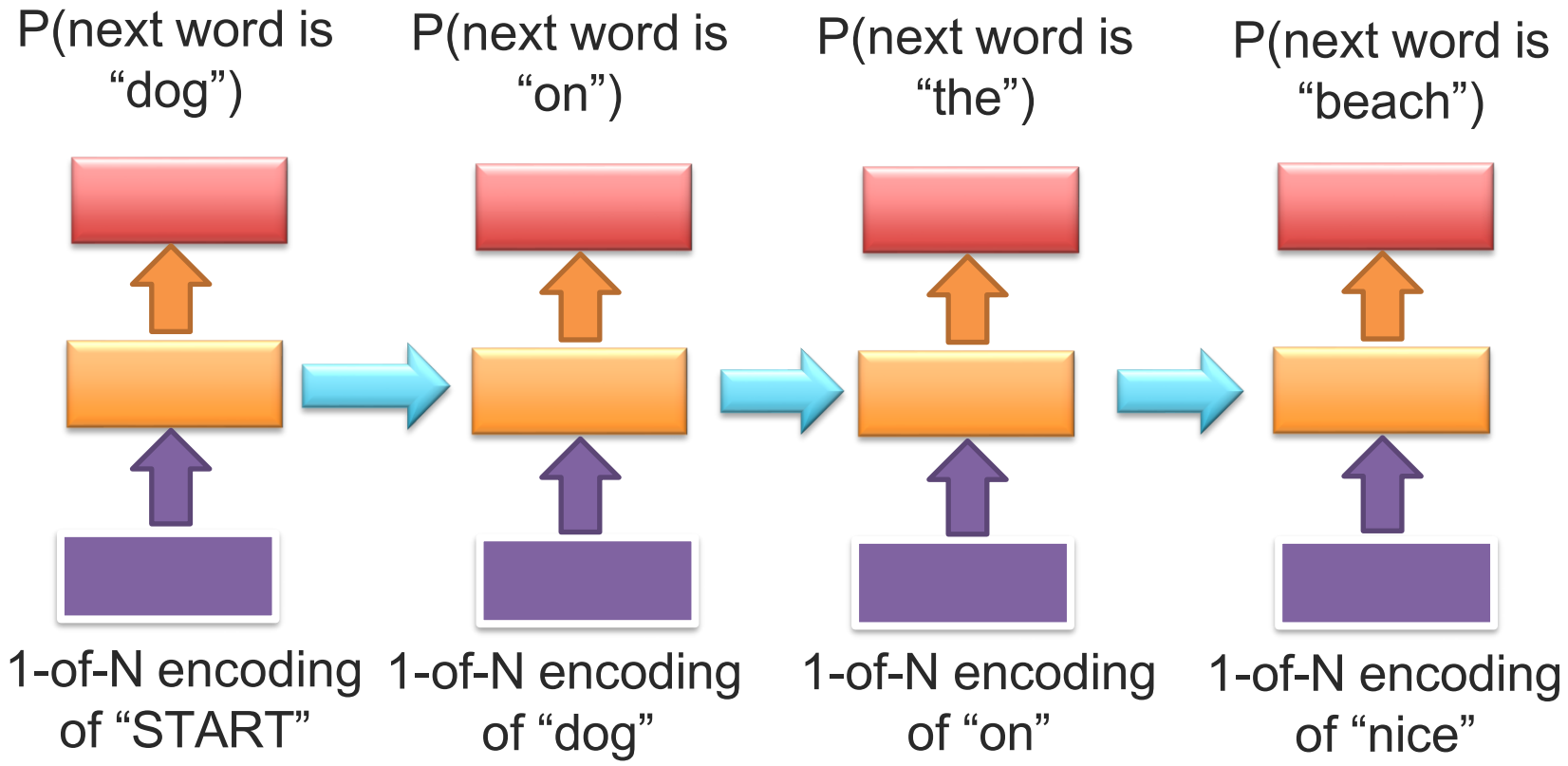
$$\arg\max_{wordsequence} P(wordsequence \mid acoustics) =$$

$$\arg\max_{wordsequence} \frac{P(acoustics \mid wordsequence) \times P(wordsequence)}{P(acoustics)}$$

$$\arg\max_{wordsequence} P(acoustics \mid wordsequence) \times P(wordsequence)$$

Language model

RNN for Language Model



Evaluating a Language Model: Cross-Entropy and Perplexity

Toy sentence and next-word probabilities

History	Target	p
<s>	→ **The**	**0.30**
The	→ **cat**	**0.12**
The cat	→ **sat**	**0.25**
The cat sat	→ **on**	**0.18**
The cat sat on	→ **the**	**0.35**
The cat sat on the	→ **mat**	**0.20**

Lower probabilities increase the loss.

Metrics

Average negative log-likelihood

$$L = -(1/N)\ \Sigma_t \log p(w_t \mid w_{<t})$$

Perplexity

$$PPL = \exp(L)$$

How to interpret it

Intuition

If PPL = 10, the model is roughly as uncertain as choosing among 10 equally likely next words on average.

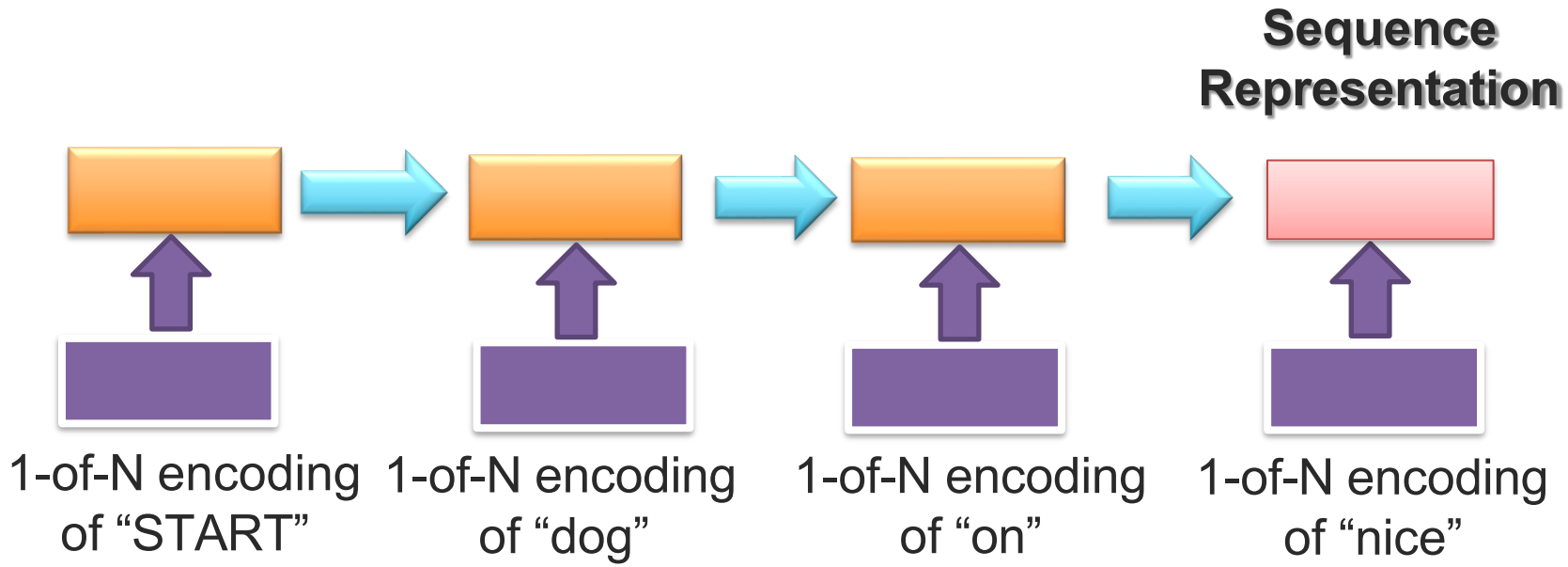
Use with care

Compare perplexity only when tokenization, vocabulary, and evaluation setup are consistent. Lower is better, but it is not the same thing as downstream task accuracy.

Example

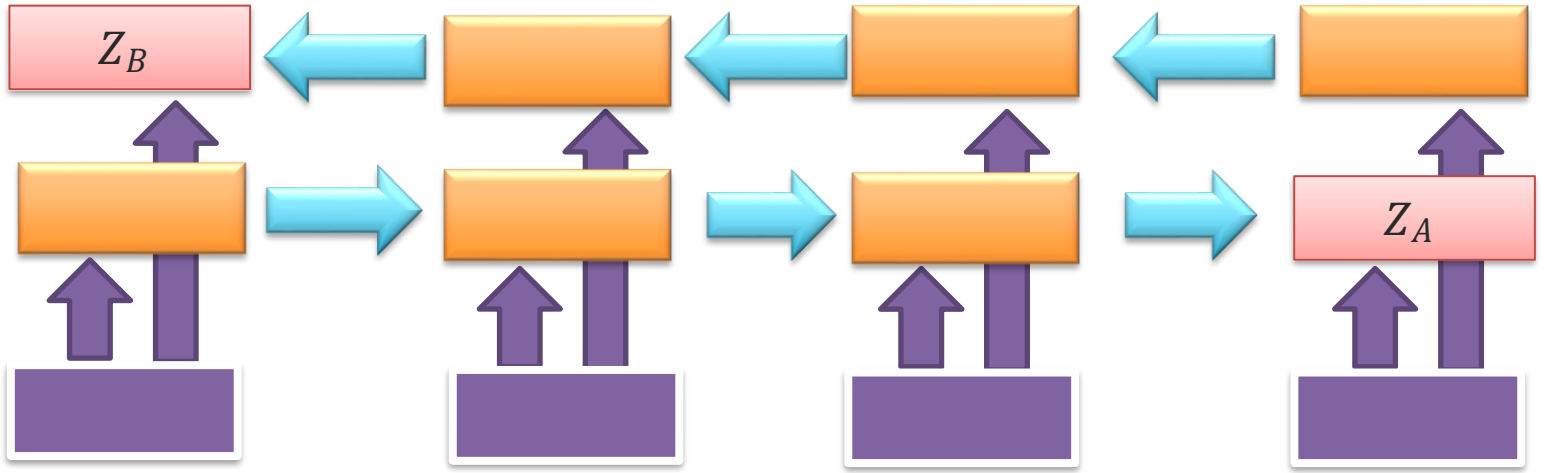
If the average NLL is 2.30, then PPL = e^2.30 ≈ 10.

RNN for Sequence Representation (Encoder)

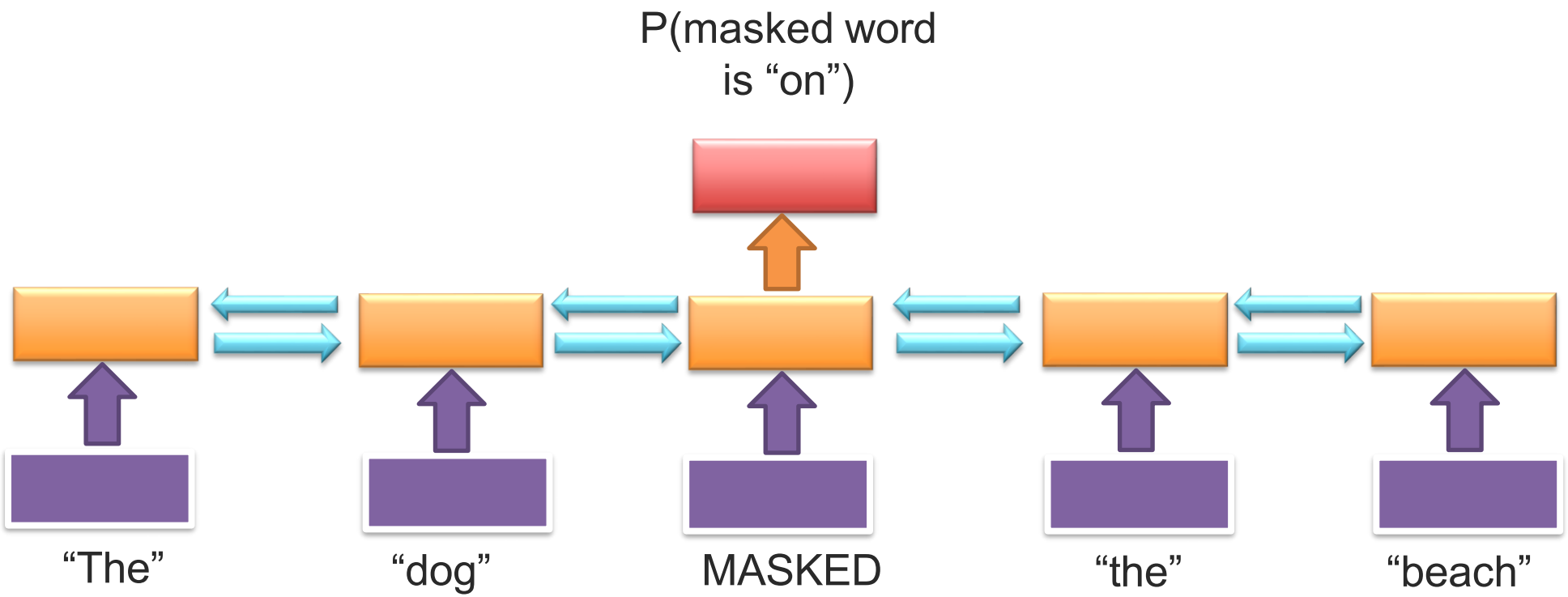


Bi-Directional RNN

Sequence Representation



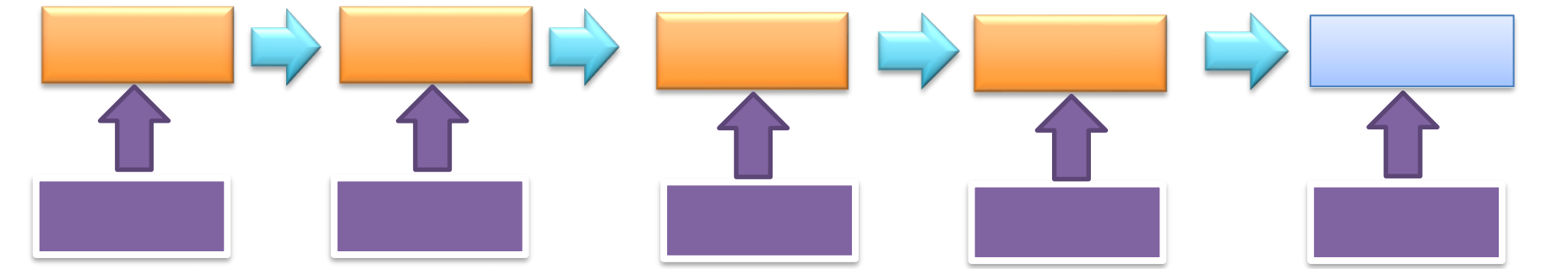
Pre-training and “Masking”



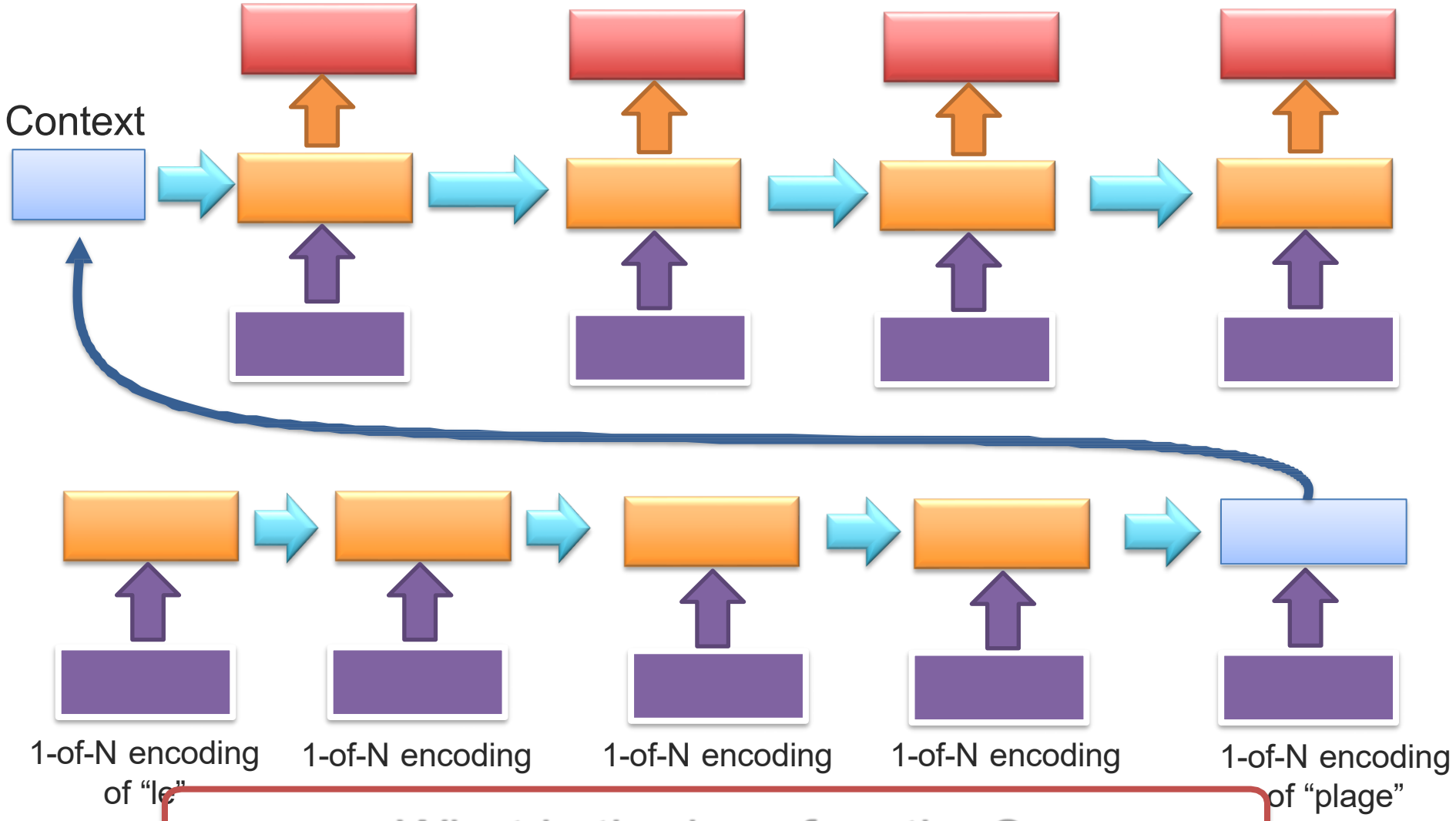
(short-lived) ELMO was a bi-directional pretrained language model

RNN-based for Machine Translation

Le chien sur la plage → The dog on the beach

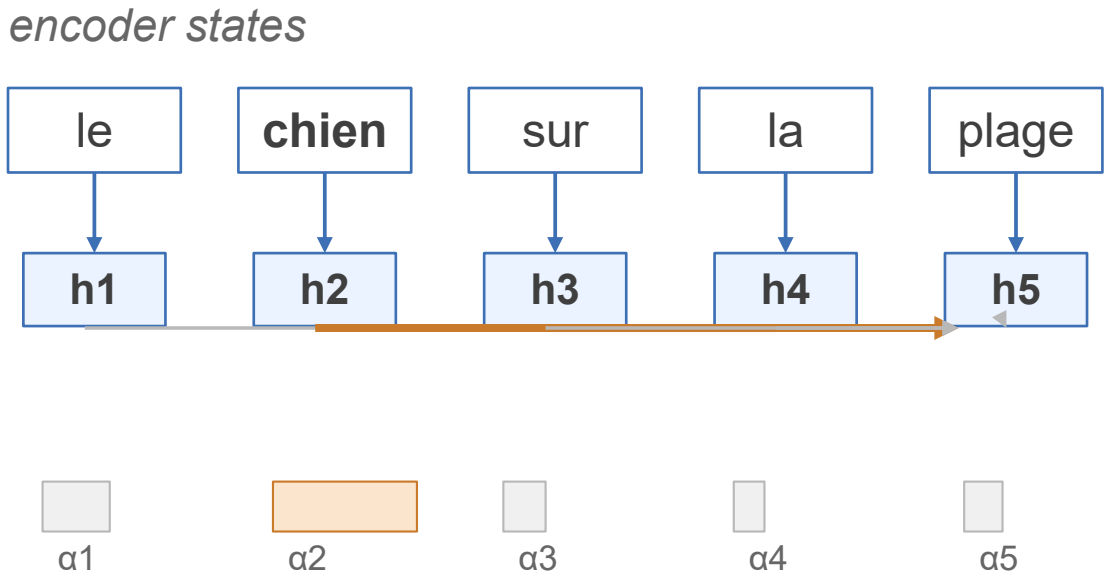


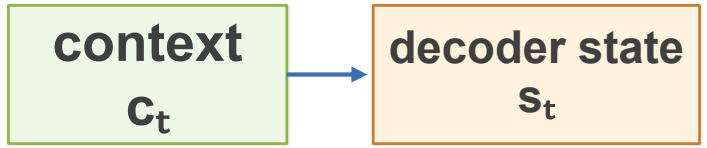
Encoder-Decoder Architecture



What is the loss function?

Attention Removes the Single-Vector Bottleneck





At the step that generates "dog," the model should focus strongly on "chien."

$$c_t = \sum_i \alpha_{ti} h_i$$

- Instead of compressing the whole source sentence into one final encoder state, attention computes a fresh context vector at every decoder step.
- The weights α tell the decoder which source words matter right now.
- This greatly improves long-sentence translation and naturally exposes alignments between source and target words.

Decoding in Seq2Seq: Greedy vs Beam Search

Greedy decoding (best local choice only)

Greedy path

Step 1: choose “the” because 0.40 > 0.35
Step 2: continue locally with “dog” at 0.35
Final sequence score: 0.40 × 0.35 = 0.14

What greedy misses

A slightly weaker first token can still lead to a stronger overall sentence once future probabilities are considered.

Toy candidates

“the dog” → 0.14
“a dog” → 0.21

Beam search (keep top B partial hypotheses)

Beam search (B = 2)

Keep both “the” and “a” after step 1.
Score expansions from both prefixes.
Return the highest-scoring full sequence among the surviving candidates.

Why it helps

Beam search is still approximate, but it is usually a much better proxy for the best full output sentence than greedy decoding.

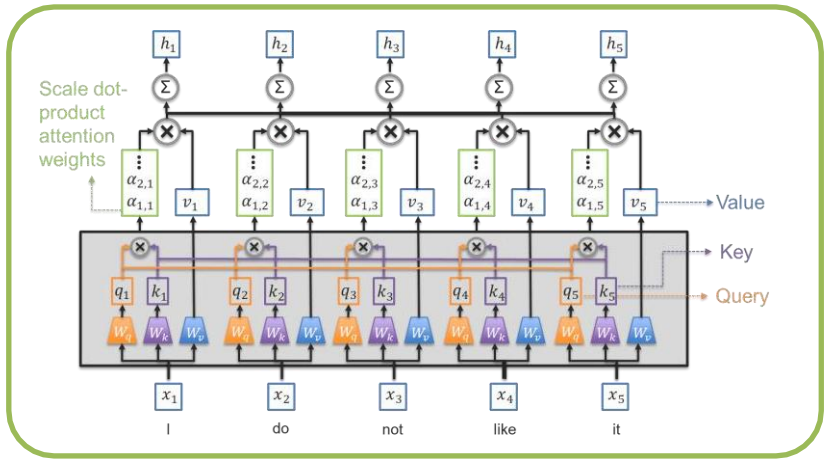
Key distinction

Training is token-level cross-entropy. Inference is a search problem over whole output sequences.

And There Are More Ways To Model Sequences…

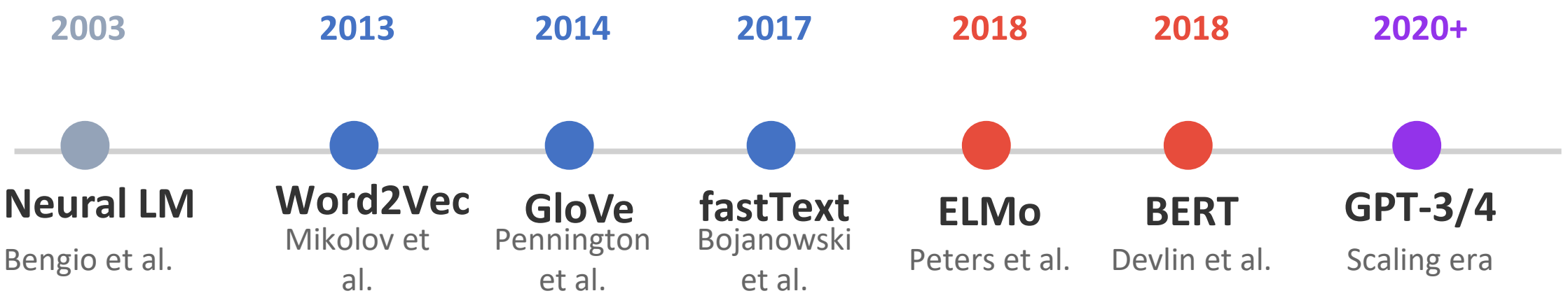
Self-attention Models

(e.g., BERT, RoBERTa)



Syntax and Language Structure

The Evolution of Language Representations



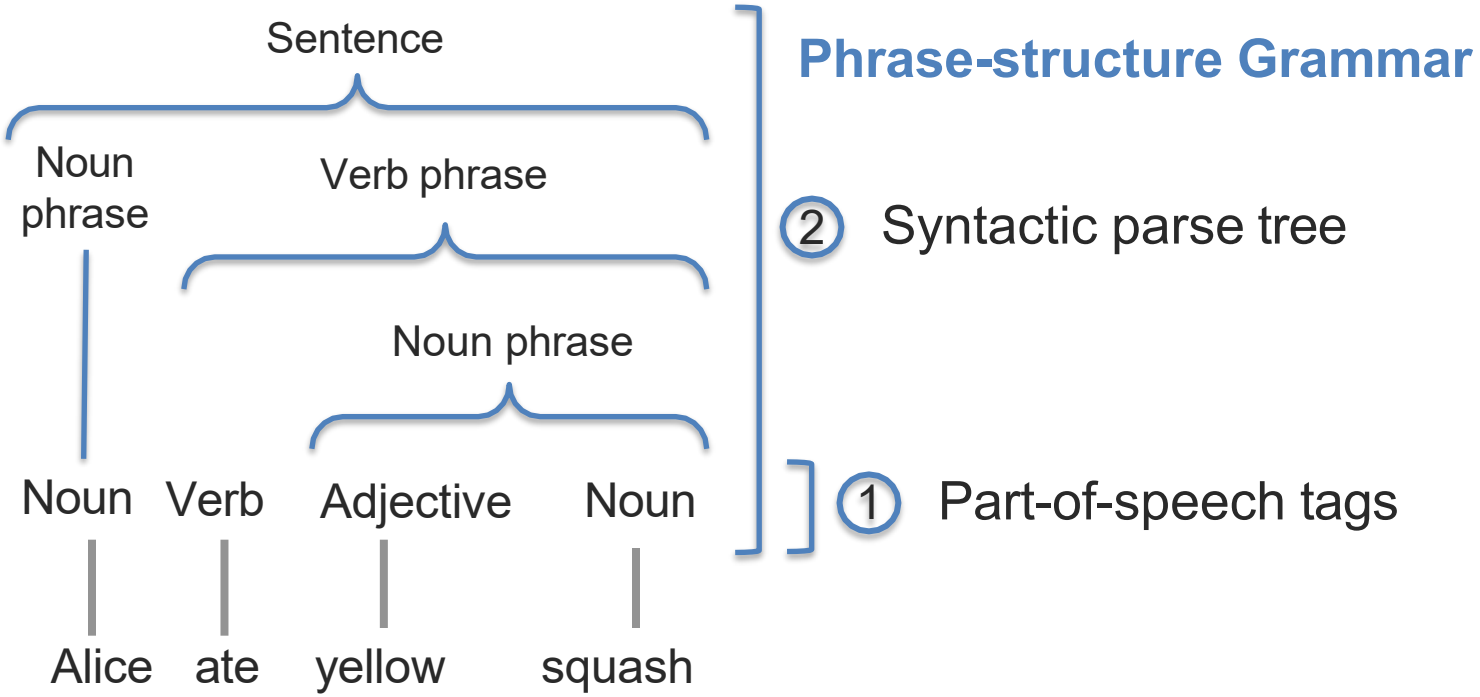
What changed?
Static: "bank" = same vector always
Contextual: "bank" ≠ in different sentences

Why Transformers won
Parallel computation (not sequential)
Direct connections via self-attention
Scales to billions of parameters

Where we are now
LLMs are giant language models
Pre-train at scale, fine-tune for tasks
Embeddings are a by-product

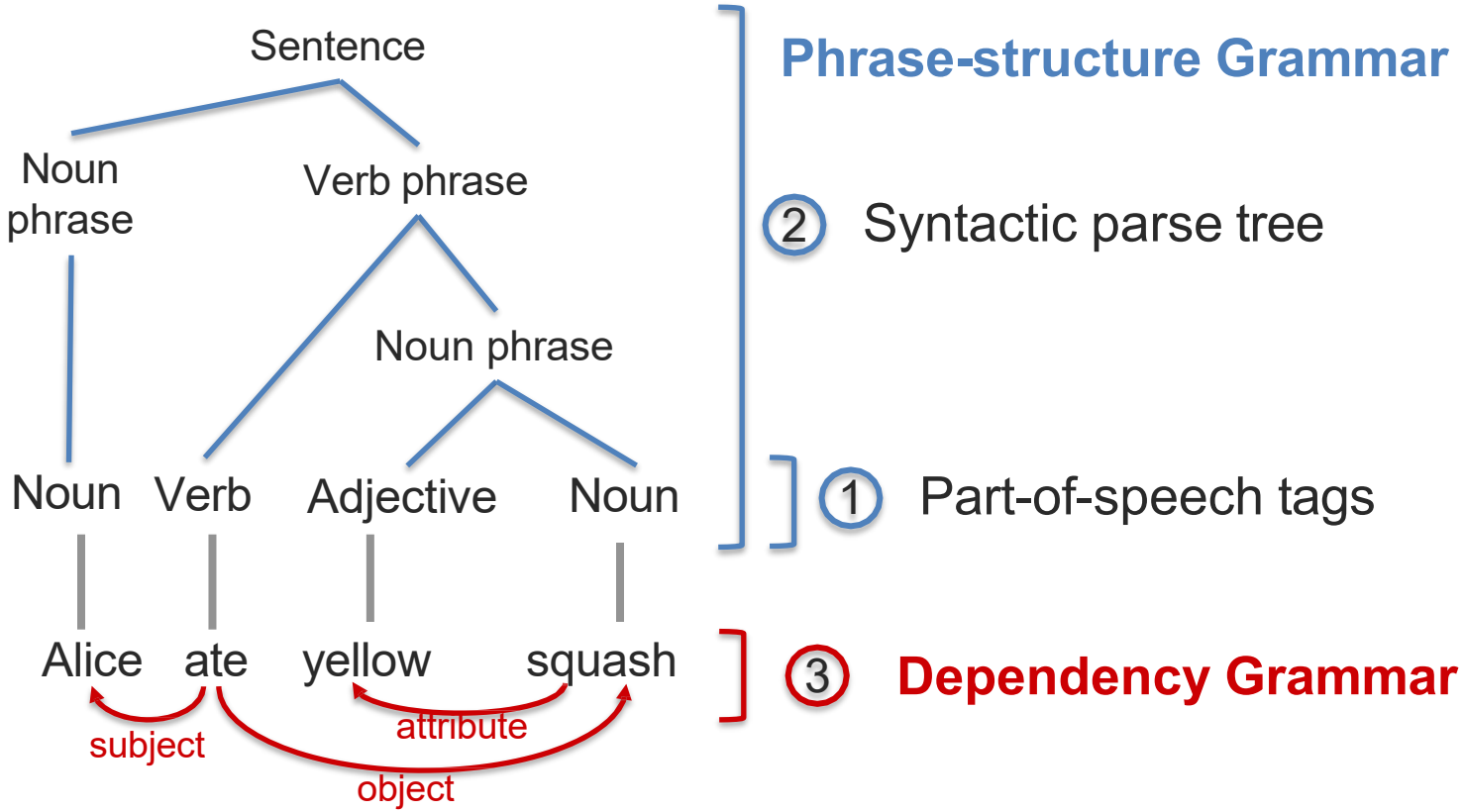
Syntax and Language Structure

What can you tell about this sentence?



Syntax and Language Structure

What can you tell about this sentence?



Ambiguity in Syntactic Parsing

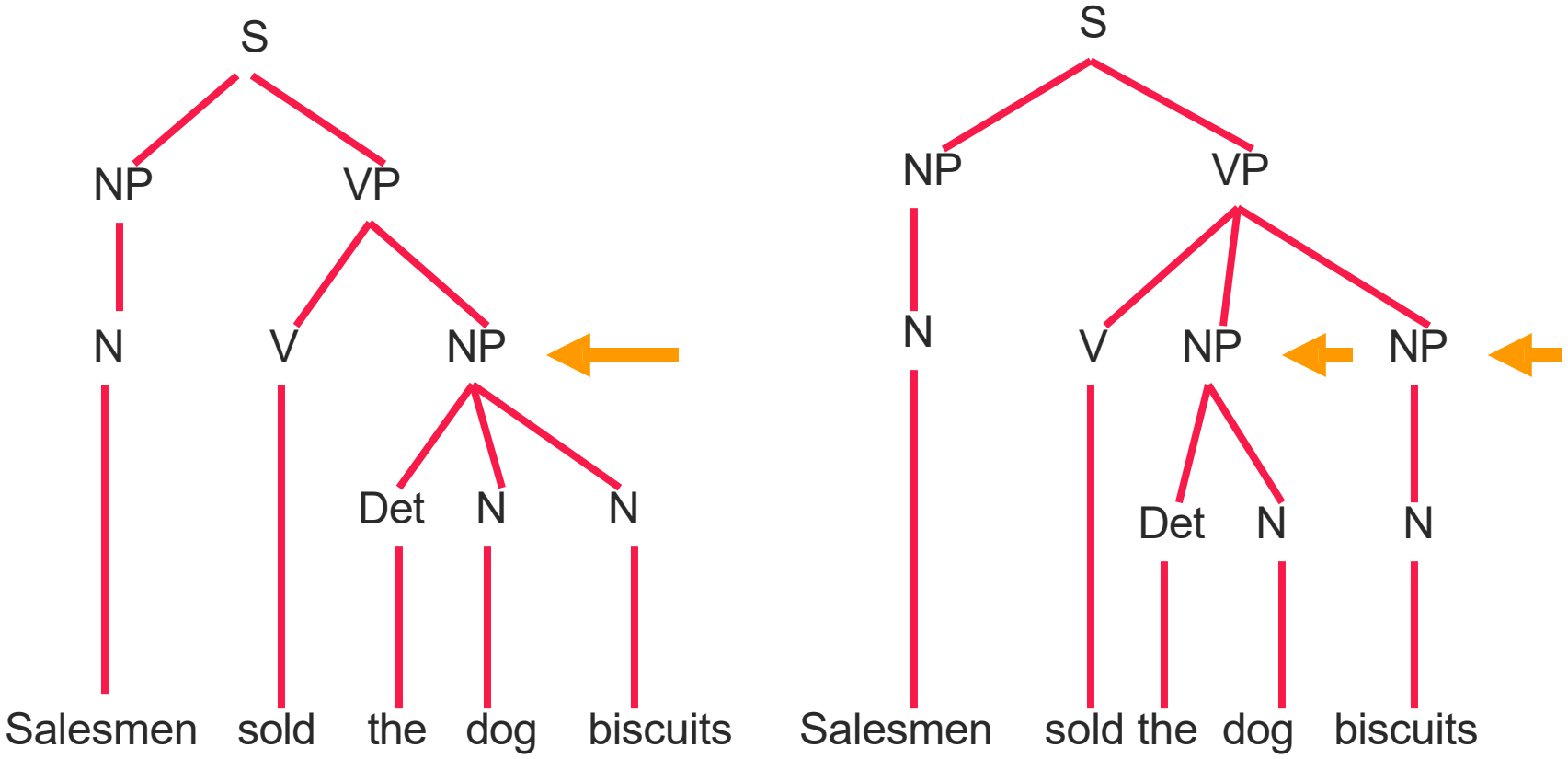
"Like" can be a verb or a preposition

- I like/VBP candy.
- Time flies like/IN an arrow.

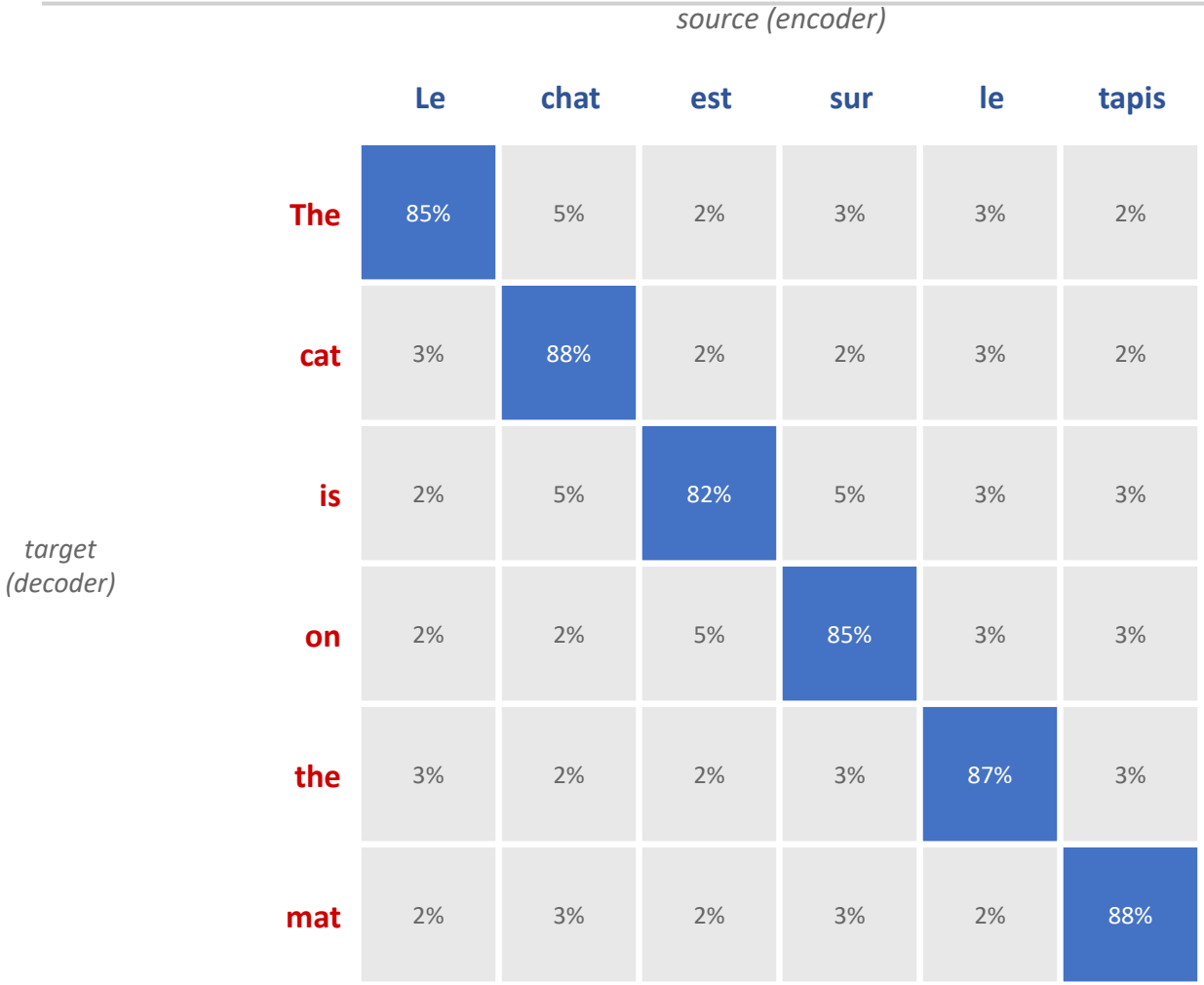
"Around" can be a preposition, particle, or adverb

- I bought it at the shop around/IN the corner.
- I never got around/RP to getting a car.
- A new Prius costs around/RB $25K.

Language Ambiguity



Attention Weights: Translation Alignment Example



Reading the heatmap

Each row shows which source words the decoder attends to when generating that target word.

When generating "cat," the model attends strongly (88%) to "chat" — the correct French translation.

Key observations:

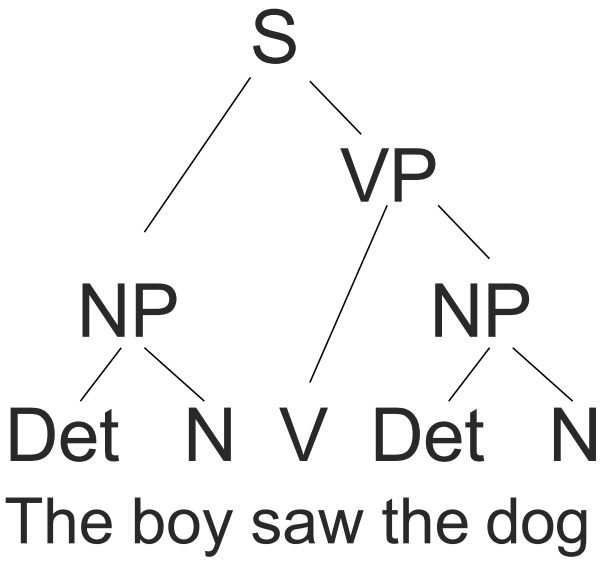
- Near-diagonal pattern: French and English word order align closely here.
- For languages with different word order (e.g., Japanese), the pattern would be far from diagonal.
- Attention weights are fully learned — no alignment supervision needed.

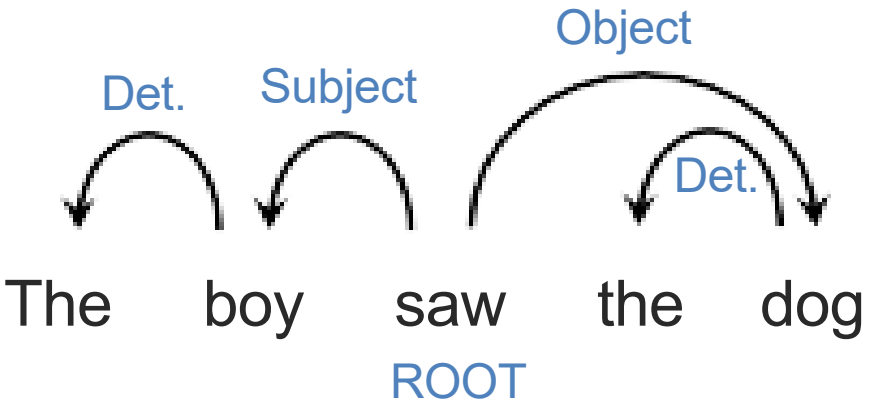
This interpretability is a major benefit of attention over fixed-context seq2seq models.

Language Syntax – Examples

Det Noun Verb Det Noun Prep Det Noun
The boy saw the dog in the park

Part of Speech tagging

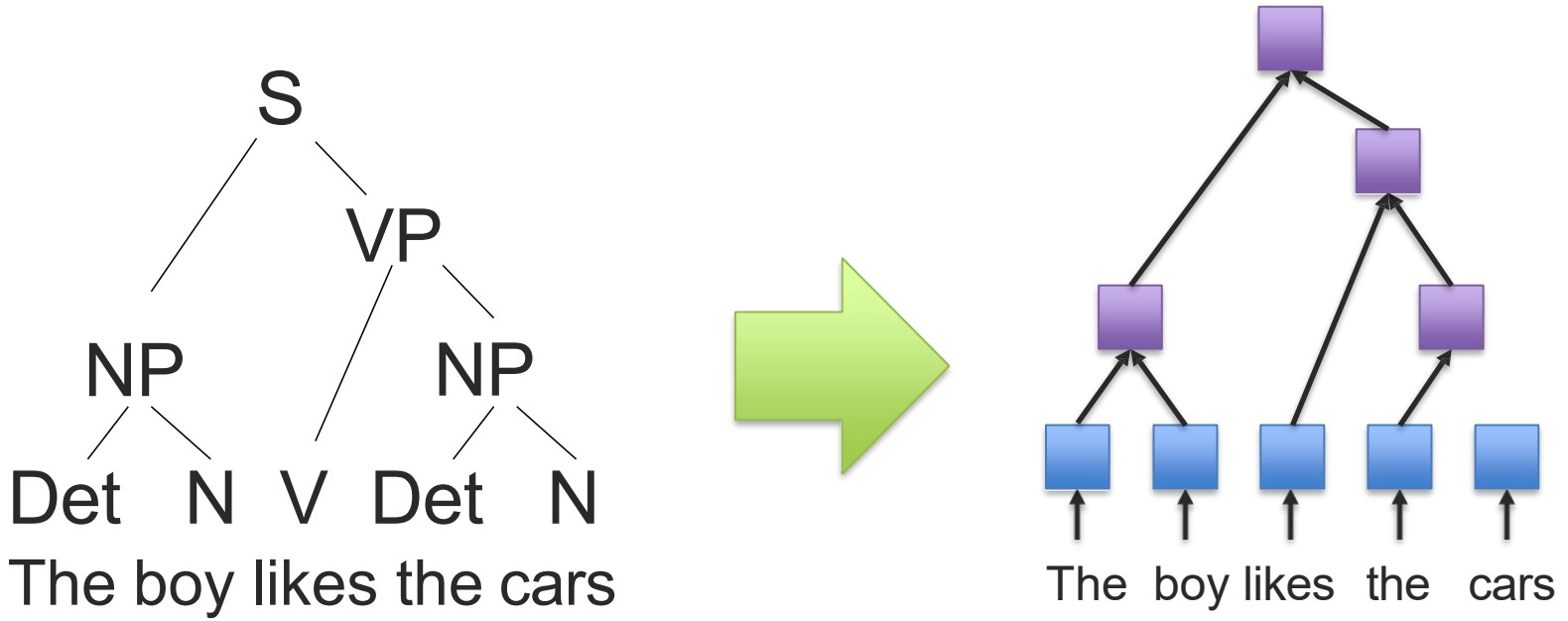




Constituency Parsing **Dependency Parsing**

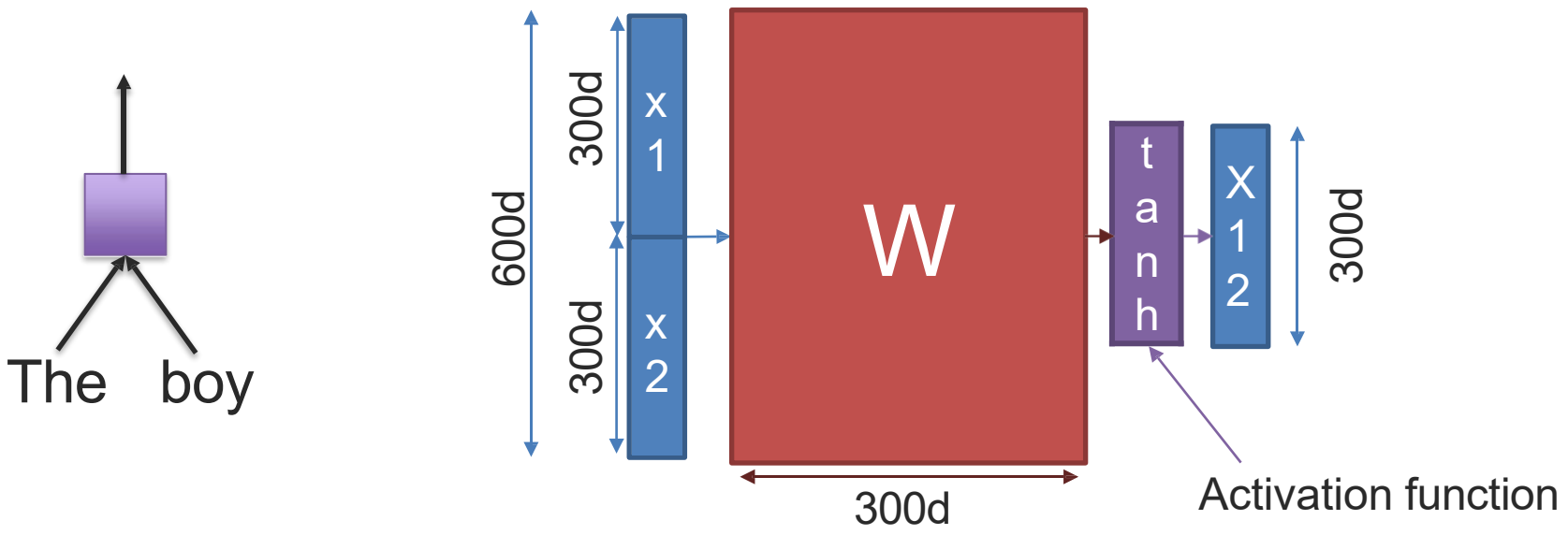
How to take advantage of syntax when modeling language with neural networks?

Tree-based RNNs (or Recursive Neural Network)



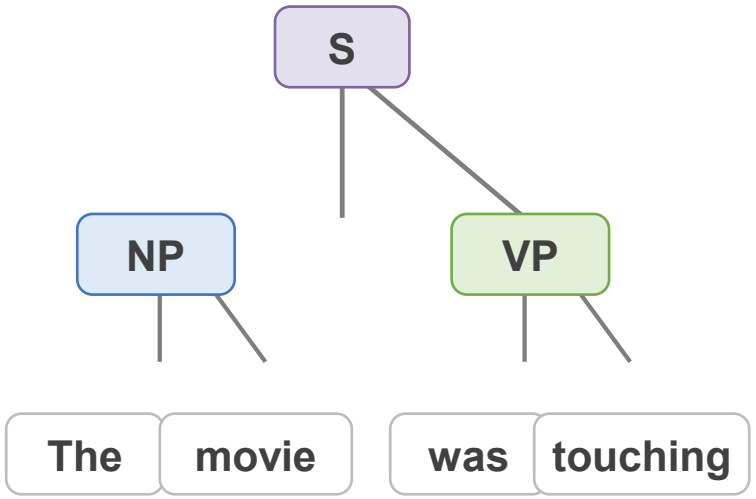
Recursive Neural Unit

Pair-wise combination of two input features

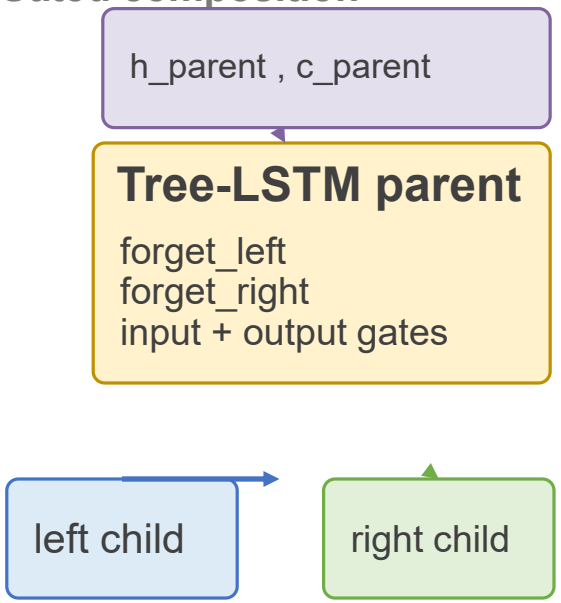


Tree-LSTM: Gating Meets Syntax-Aware Composition

Parse-tree structure



Gated composition



Why it matters

- Separate forget gates let the parent keep different amounts of information from each child.
- The cell state gives Tree-LSTM better gradient flow than a plain TreeRNN.
- Strong when syntax matters: sentiment over long phrases, relation extraction, and low-resource settings.
- Trade-off: you need a parse tree, so parser errors can propagate.

Core update

$$c\_p = f\_l \odot c\_l + f\_r \odot c\_r + i \odot \tilde{c}\_p, \quad h\_p = o \odot \tanh(c\_p)$$

Key Takeaways

Words

- One-hot vectors are simple but sparse and semantically blind.
- Distributional learning turns context into dense embeddings.
- Static spaces are useful, but they can encode social bias from data.

similar context → similar vector

Sequences

- RNNs share parameters across time and model word order.
- Vanilla RNNs struggle with long-range dependencies because of vanishing/exploding gradients.
- LSTM/GRU use gates; attention lets the model look back selectively.

memory + gating + attention

Structure

- Language is hierarchical, not just sequential.
- Constituency and dependency parses expose useful grammatical relations.
- TreeRNNs compose representations along syntax-aware structures.

how you compose matters

Resources

Core NLP Tools for Analysis and Parsing

General-purpose NLP pipelines

- spaCy — fast tokenization, POS tagging, dependency parsing, and rule-based pipelines.
- Stanza — Stanford's neural NLP toolkit with multilingual support.
- Berkeley Neural Parser — strong constituency parser for English syntax.

spaCy: https://spacy.io

Stanza: https://stanfordnlp.github.io/stanza

Berkeley parser: https://github.com/nikitakit/self-attentive-parser

When to use what

Use spaCy or Stanza when you need a practical pipeline quickly.

Use a dedicated parser when syntax quality matters for your experiment.

Remember: neural encoders often absorb some syntax implicitly, but explicit parsers still matter for analysis and structure-aware models.

- Recommended classroom workflow: tokenize → inspect POS/dependencies → compare what your neural model learns.

Embedding and Model Resources

Static representations

- Word2Vec — local prediction objective; classic dense word vectors.
- GloVe — global co-occurrence factorization.
- fastText — subword-aware embeddings that help with morphology and OOV words.

fastText: https://fasttext.cc

GloVe: https://nlp.stanford.edu/projects/glove

Static embeddings give one vector per word type.

Contextual representations

- ELMo — contextualized embeddings from bidirectional language modeling.
- BERT / RoBERTa — Transformer encoders trained with masked language modeling.
- Hugging Face Transformers — easiest practical gateway to pretrained modern NLP models.

Transformers: https://huggingface.co/docs/transformers

Model hub: https://huggingface.co/models

Contextual models produce different vectors for the same word in different sentences.

Lexicons and Practical Notes

Useful lexicon resources

- LIWC — hand-built categories for function words, affect, social language, cognition, drives, time orientation, and more.
- General Inquirer — early broad-coverage semantic categories.
- OpinionFinder and SentiWordNet — lexicons tuned for sentiment and subjectivity.

Lexicons are interpretable and easy to audit, but they are brittle outside their design domain.

LIWC: https://liwc.app

Course/project guidance

LIWC is commercial software. For HUST projects, coordinate with the instructor or course staff before planning to rely on it.

- Lexicons are best used as baselines, interpretable features, or analysis tools—not as the only representation.
- Compare lexicon features with neural representations to understand what each captures well or misses.
- This is especially useful in low-resource settings or when interpretability matters.

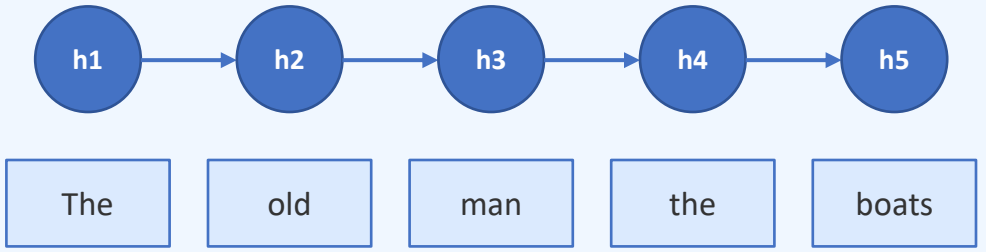
Key References

- Elman (1990). Finding Structure in Time.
- Hochreiter & Schmidhuber (1997). Long Short-Term Memory.
- Mikolov et al. (2013). Efficient Estimation of Word Representations in Vector Space.
- Pennington et al. (2014). GloVe: Global Vectors for Word Representation.
- Sutskever et al. (2014). Sequence to Sequence Learning with Neural Networks.
- Bahdanau et al. (2015). Neural Machine Translation by Jointly Learning to Align and Translate.
- Bolukbasi et al. (2016). Man is to Computer Programmer as Woman is to Homemaker?
- Vaswani et al. (2017). Attention Is All You Need.
- Peters et al. (2018). Deep contextualized word representations.
- Socher et al. (2011). Parsing Natural Scenes and Natural Language with Recursive Neural Networks.

These papers give a clean historical path: embeddings → gated RNNs → seq2seq + attention → contextual models.

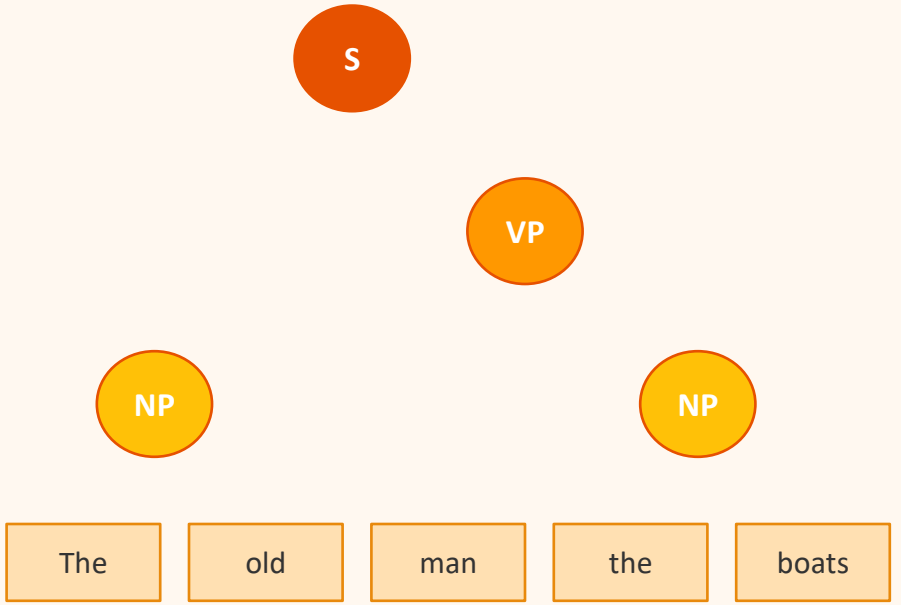
Sequential RNN vs TreeRNN: How Composition Differs

Sequential RNN



Left-to-right processing.
"man" and "boats" are 3 steps apart.
Gradient must flow through all steps.

TreeRNN (Recursive)



Bottom-up composition along parse tree.
"man" (verb) directly composes with "boats."
Shorter gradient paths = better learning.